# PI_AI_Curs_02

# 2.1

Excelent! Am ajuns la punctul de cotitură în înțelegerea noastră, făcând tranziția de la modelele statistice liniare la limbajul rețelelor neuronale. Iată ghidul dumneavoastră de studiu complet, bazat pe secțiunea 2.1.

---

## Ghid de Studiu Complet: 2.1 De la Modele Liniare la Rețele Neuronale Profunde

### 1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Această lecție marchează trecerea de la familia simplă a modelelor liniare (discutate în Capitolul 1) la familia mult mai bogată și mai expresivă a **rețelelor neuronale (*neural networks*)**. Obiectivul principal este reformularea modelelor liniare cunoscute (precum Regresia Liniară și Regresia Softmax) folosind **notația stratificată (*"layer" notation*)** a *Deep Learning*-ului. Prin această reformulare, se demonstrează că modelele liniare sunt, de fapt, **rețele neuronale cu un singur strat (*single-layer neural networks*)**. De asemenea, este introdusă structura biologică a neuronului ca inspirație istorică pentru unitățile artificiale, deși se subliniază că *Deep Learning*-ul modern se bazează mai mult pe matematică și statistică.

**Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?**

1. **Echivalența Fundamentală:** Atât Regresia Liniară, cât și Regresia Softmax pot fi vizualizate ca **rețele neuronale cu un singur strat**.
2. **Structura Stratului:** Stratul de ieșire (*Output layer*) al acestor modele este un **strat complet conectat (*fully-connected layer*)** sau **strat dens (*dense layer*)**, deoarece fiecare intrare este conectată la fiecare ieșire.
3. **Reprezentarea Datelor Categorice:** Pentru problemele de clasificare generală fără o ordonare naturală a claselor (ex: "pisică", "câine"), etichetele trebuie reprezentate folosind **codificarea *one-hot*** .
4. **Costul Parametrizării:** Costul unei rețele complet conectate crește cu $O(dq)$ (unde $d$ este numărul de intrări și $q$ este numărul de ieșiri), ceea ce devine problematic în practică.

---

### 2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

## 2.1 De la Modele Liniare la Rețele Neuronale Profunde

- **Fundamente:** Modelele liniare discutate în Capitolul 1 acoperă doar o familie restrânsă de modele, în timp ce **rețelele neuronale** acoperă o familie mult mai bogată.
- **Vizualizare:** Diagramele sunt utilizate în *Deep Learning* pentru a vizualiza modelele și pentru a evidenția **tiparul de conectivitate** (cum se leagă intrările de ieșiri), dar nu și valorile specifice ale ponderilor sau bias-urilor.
- **Regresia Liniară ca Rețea Neuronală:** Regresia liniară este reprezentată ca o rețea neuronală cu un singur strat (Figura 1).
  - Numărul de intrări ($x_1, \ldots, x_d$) (sau dimensionalitatea caracteristicilor) în stratul de intrare este $d$.
  - Numărul de ieșiri în stratul de ieșire este 1 ($o_1$).
  - În mod convențional, stratul de intrare nu este luat în considerare la numărarea straturilor. Prin urmare, Regresia Liniară are **1 strat**.
  - Acest model constă dintr-un **singur neuron artificial** sau o **rețea neuronală cu un singur strat**.
- **Strat Complet Conectat (*Fully-connected layer*):** Deoarece, în regresia liniară, fiecare intrare este conectată la fiecare ieșire (care este un singur output), această transformare este numită **strat complet conectat** sau **strat dens (*dense layer*)**.

## Inspirație Biologică (Neuronul Real)

- Regresia liniară (inventată în **1795**) este anterioară neuroștiinței computaționale.
- Modelele de neuroni artificiali au început să fie dezvoltate de **Warren McCulloch** și **Walter Pitts**.
- Un neuron biologic (Figura 2) este descris astfel:
  - Informația ($x_i$) este primită în **dendrite**.
  - Informația este ponderată de **greutăți sinaptice** $w_i$.
  - Intrările ponderate sunt agregate în **nucleu (CPU)** sub forma unei sume ponderate: $y = \sum_{i=1}^{d} x_i w_i + b$.
  - Informația este apoi procesată în **axon**, adesea după o prelucrare neliniară $\sigma(y)$.
- Cercetarea modernă în *Deep Learning* astăzi se inspiră mai mult din **matematică, statistică și informatică**, deși conceptul de bază provine din studiul neuronilor biologici.

## Regresia Softmax ca Rețea Neuronală

- Regresia Softmax poate fi și ea încadrată ca o rețea neuronală.
- **Exemplu:** Dacă avem o imagine de 2x2 grayscale (4 caracteristici: $x_1, x_2, x_3, x_4$) și 3 categorii ("cat", "chicken", "dog").
- **Codificarea Etichetelor:** Deoarece nu există o ordine naturală între categorii, se folosește **codificarea *one-hot*** .
  - "cat" corespunde vectorului $(1, 0, 0)$.

- **Calculul Scorului (Logits):** Pentru a estima probabilitățile condiționate, este nevoie de un model cu ieșiri multiple (una per clasă).
  - Avem 4 caracteristici și 3 categorii de ieșire, deci sunt necesari **12 scalari** pentru ponderi ($W_{ij}$) și **3 scalari** pentru bias-uri ($b_1, b_2, b_3$).
  - Se calculează 3 logiți ($o_1, o_2, o_3$) pentru fiecare intrare, fiecare fiind o funcție afină:

$$O_1 = x_1W_{11} + x_2W_{12} + x_3W_{13} + x_4W_{14} + b_1$$
$$O_2 = x_1W_{21} + x_2W_{22} + x_3W_{23} + x_4W_{24} + b_2$$
$$O_3 = x_1W_{31} + x_2W_{32} + x_3W_{33} + x_4W_{34} + b_3$$

- **Softmax ca NN cu un singur strat:** Această structură este, de asemenea, o **rețea neuronală cu un singur strat** (Figura 3). Stratul de ieșire este un **strat complet conectat**.

## Forma Vectorială și Predicția

- **Forma Vectorială (Per Instanță):** Vectorul de scoruri (logits) se scrie compact: $\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$.
- **Forma Vectorială (Batch):** Pentru o matrice de design $\mathbf{X} \in \mathbb{R}^{n \times d}$ (n exemple, d intrări) și o matrice de ponderi $\mathbf{W} \in \mathbb{R}^{d \times q}$ (q ieșiri) și bias $\mathbf{b} \in \mathbb{R}^{1 \times q}$:

$$\mathbf{O} = \mathbf{X}\mathbf{W} + \mathbf{b}$$

.

- **Probabilități:** Ieșirile $\mathbf{O}$ sunt transformate în probabilități $\hat{\mathbf{Y}}$ prin funcția Softmax:

$$\hat{\mathbf{Y}} = \mathrm{softmax}(\mathbf{O})$$

.

$$\hat{y}_j = \frac{\exp(o_j)}{\sum_{i=1}^{q} \exp(o_i)}$$

.

- **Decizia:** Softmax-ul nu schimbă ordinea scorurilor. Decizia finală este dată de **argmax** pe scoruri sau probabilități:

$$\arg\max_j \hat{y}_j = \arg\max_j o_j$$

.

- **Costul Parametrizării:** Pentru un strat complet conectat cu $d$ intrări și $q$ ieșiri, costul este $O(dq)$. Aceasta poate fi prohibitv de mare în practică.

---

# 3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

### Termen Tehnic: Strat Complet Conectat (Fully-Connected Layer)

- **Explicație Simplificată (ELI5):** Imaginați-vă o sală de clasă. În primul rând (stratul de intrare) stau elevii ($x_1, x_2, \ldots$). În rândul de la tablă (stratul de ieșire) stau profesorii ( $o_1, o_2, \ldots$). Un strat este *complet conectat* dacă fiecare elev $x_i$ vorbește direct cu **fiecare** profesor $o_j$.
- *Asta se întâmplă deoarece:* Această conectivitate totală este cea care implementează suma ponderată liniară ($o_j = \sum x_i w_{ij} + b_j$), care este baza modelului liniar.

### Concept Dificil: Codificarea One-Hot

- **Context:** De ce nu folosim pur și simplu numere întregi (ex: Pisică=1, Pui=2, Câine=3) pentru etichete?
- *Asta se întâmplă deoarece:* Folosirea numerelor întregi introduce o **relație de ordine artificială**. Modelul ar interpreta că $3 > 2 > 1$, sugerând o ierarhie sau o distanță numerică între clase, ceea ce este fals pentru categorii nominale ("cat" nu este "mai mult" decât "chicken").
- **One-Hot (Soluția):** Codificarea *one-hot* (vectori cu o singură componentă setată la 1) tratează fiecare categorie ca pe o axă independentă în spațiu, asigurând că nu se impune nicio relație de ordine.

### Costul Parametrizării $O(dq)$

- **Clarificare:** Costul parametrizării se referă la numărul total de ponderi ($W$) și bias-uri ($b$) pe care modelul trebuie să le învețe.
- $d$: Numărul de intrări.
- $q$: Numărul de ieșiri.
- Numărul de ponderi este $d \times q$ (matricea $\mathbf{W} \in \mathbb{R}^{d \times q}$).
- Numărul de bias-uri este $q$.
- Costul este aproximativ $d \times q$. Dacă o imagine are $d = 1.000.000$ de pixeli și avem $q = 1000$ de clase, $10^6 \times 10^3 = 1$ miliard de parametri doar pentru acest prim strat. Acest cost mare face modelul *fully-connected* nepractic pentru date cu dimensionalitate mare, cum ar fi imaginile.

---

## 4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Liniaritate/Neliniaritate:** Deși Softmax Regression folosește funcția **Softmax (care este neliniară)** pentru a calcula probabilitățile, ieșirile sale (logits) sunt determinate de o **transformare afină** a caracteristicilor de intrare. Din acest motiv, modelul Softmax este încă considerat **liniar** în esența sa de bază, deoarece scorurile sunt calculate liniar.

- **Trecerea la MLP (Implicația Logică):** Limitarea fundamentală a modelelor liniare/single-layer (Linear Regression, Softmax Regression) este capacitatea lor expresivă limitată și costul ridicat $O(dq)$. **Implicația majoră** este că, pentru a obține o familie de modele mai bogată, trebuie să "coasem împreună" mai multe unități (neuroni) cu o conectivitate și un algoritm de învățare adecvat. Aceasta este motivația directă pentru introducerea **Perceptronului Multi-Strat (Multilayer Perceptron - MLP)** și a rețelelor neuronale profunde.
- **Vizualizarea ca Bază pentru DL:** Înțelegerea faptului că Regresia Softmax este o rețea cu un singur strat și este descrisă de un strat **fully-connected** oferă vocabularul de bază pentru a descrie arhitecturile neuronale mai complexe care vor urma.

---

# 5. EXERCIȚII ȘI REZOLVARE

Vom folosi notația matricială și de parametri introdusă în secțiunea 2.1 și structura de rezolvare din sursă.

## Exercițiul F.3: Parametrizarea Softmax Regression

Considerăm un model de Regresie Softmax pentru o problemă cu $d = 5$ caracteristici de intrare și $q = 4$ clase de ieșire.

1. Specificați dimensiunea matricei de ponderi $\mathbf{W}$ și a vectorului de bias $\mathbf{b}$.
2. Calculați costul total al parametrilor (greutăți și bias-uri) pentru acest strat complet conectat.
3. Calculați costul parametrilor dacă numărul de caracteristici ar crește la $d = 1000$, menținând $q = 4$.

**Rezolvare F.3:**

1. **Dimensiuni:** Conform surselor, Matricea de design $\mathbf{X}$ are dimensiunea $n \times d$, iar ponderile $\mathbf{W}$ au dimensiunea $d \times q$. Bias-ul $\mathbf{b}$ are dimensiunea $1 \times q$.
   - $\mathbf{W} \in \mathbb{R}^{d \times q}$: $\mathbf{W} \in \mathbb{R}^{5 \times 4}$.
   - $\mathbf{b} \in \mathbb{R}^{1 \times q}$: $\mathbf{b} \in \mathbb{R}^{1 \times 4}$.
2. **Costul Total (d=5, q=4):**
   - Număr de greutăți (Weight parameters): $d \times q = 5 \times 4 = 20$.
   - Număr de bias-uri (Bias parameters): $q = 4$.
   - Total parametri: $20 + 4 = \mathbf{24}$.
3. **Costul Total (d=1000, q=4):**
   - Număr de greutăți: $d \times q = 1000 \times 4 = 4000$.
   - Număr de bias-uri: $q = 4$.
   - Total parametri: $4000 + 4 = \mathbf{4004}$.

- *Implicație:* Acest exercițiu ilustrează cum costul parametrizării $O(dq)$ crește liniar cu dimensionalitatea intrării, dar menține costul gestionabil dacă $d$ este mare, dar $q$ este mic.

### Exercițiul F.4: Structura Softmax și Scorul Liniar (Logits)

Considerăm un caz simplu de clasificare binară cu Softmax (clase $q = 2$). Modelul folosește două caracteristici $x_1, x_2$ și bias-uri.

1. Scrieți ecuația scalară (extinsă) pentru scorul clasei 1 ($o_1$) și scorul clasei 2 ($o_2$), folosind notația $w_{ij}$ și $b_j$ din sursă.
2. Explicați cum ar trebui reprezentată eticheta țintă $y$ pentru o instanță care aparține Clasei 2, folosind codificarea *one-hot*.

**Rezolvare F.4:**

1. **Ecuațiile Scorului (Logits):** Avem $d = 2$ intrări $(x_1, x_2)$ și $q = 2$ ieșiri $(o_1, o_2)$. Ponderile vor fi notate $W_{ij}$ și bias-urile $b_j$:
   - Scorul Clasei 1:
   $$O_1 = x_1 W_{11} + x_2 W_{12} + b_1$$
   - Scorul Clasei 2:
   $$O_2 = x_1 W_{21} + x_2 W_{22} + b_2$$
2. **Codificarea One-Hot:** Pentru $q = 2$ clase, vectorul *one-hot* are 2 componente. Dacă instanța aparține Clasei 2, componenta corespunzătoare acestei clase este setată la 1, iar restul la 0.
$$y = [0\ 1]$$

# 2.2

Absolut! Trecem acum la **Perceptronii Multistrat (Multilayer Perceptrons - MLP)**, prima noastră rețea cu adevărat "profundă" (*deep*). Această lecție explică de ce simpla stivuire a straturilor liniare nu funcționează și cum adăugarea neliniarității (funcțiilor de activare) ne permite să modelăm funcții de o complexitate mult mai mare.

---

# Ghid de Studiu Complet: 2.2 Perceptronii Multistrat (Multilayer Perceptrons - MLP)

## 1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Această lecție introduce **Perceptronul Multistrat (MLP)**, cel mai simplu model de rețea neuronală profundă, care constă din mai multe straturi de neuroni complet conectați. Punctul

cheie de plecare este înțelegerea faptului că, deși stivuirea a două sau mai multor transformări afine (liniare plus bias) adaugă parametri suplimentari, **rezultatul rămâne o singură transformare afină**, oferind **zero câștig de expresivitate**. Pentru a depăși limitarea liniarității, este necesară introducerea unui **ingredient cheie**: o **funcție de activare neliniară** (non-linear activation function) $\sigma$ aplicată unităților din straturile ascunse. Prin stivuirea straturilor complet conectate împreună cu neliniarități, MLP-urile pot captura **interacțiuni complexe** între intrări, devenind **aproximatori universali (*universal approximators*)**.

**Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?**

1. **Definiție MLP:** Este cea mai simplă rețea profundă, constând din **multiple straturi de neuroni** complet conectați, unde fiecare strat este conectat la stratul de dedesubt și cel de deasupra.
2. **Stratul Ascuns (Hidden Layer):** Un MLP conține unul sau mai multe **straturi ascunse** ( $L-1$ straturi, unde $L$ este numărul total de straturi), care acționează ca o **reprezentare internă (hidden representations)** a datelor.
3. **Eșecul Liniarității:** O compunere de transformări afine (liniar + bias) este ea însăși o transformare afină; prin urmare, stivuirea straturilor complet conectate **fără neliniaritate nu adaugă putere de modelare**.
4. **Ingredientul Cheie:** Pentru a realiza arhitecturi multistrat expresive, este obligatoriu să se aplice o **funcție de activare neliniară** $\sigma$ (ex: ReLU, Sigmoid, tanh) la ieșirea fiecărei unități ascunse.
5. **ReLU (Rectified Linear Unit):** Este cea mai populară funcție de activare datorită simplității și performanței bune. Formula sa este $\mathrm{ReLU}(x) = \max(x, 0)$.

---

# 2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

## 2.2 Perceptronii Multistrat (MLP) și Structura Arhitecturală

- MLP-urile sunt cele mai simple rețele profunde.
- Ele constau din multiple straturi de neuroni, fiecare complet conectat la neuronii din stratul anterior (de la care primesc input) și la cei din stratul următor (pe care îi influențează).
- **Structura MLP:**
  - **Stratul de Intrare (*Input layer*):** Primește caracteristicile $\mathbf{x}$.
  - **Straturi Ascunse (*Hidden layers*):** Primele $L-1$ straturi. Ieșirile acestor straturi sunt numite **reprezentări ascunse (*hidden representations*)** sau variabile ascunse.
  - **Stratul de Ieșire (*Output layer*):** Ultimul strat, care acționează ca un **predictor liniar (*linear predictor*)** (regresor liniar pentru regresie sau Softmax/Logistic pentru clasificare).

- **Numărarea Straturilor:** Deoarece stratul de intrare nu implică niciun calcul, numărul de straturi se referă la straturile de calcul (ascunse și de ieșire). De exemplu, un MLP cu un singur strat ascuns are **2 straturi**.

## Calculul în Straturile Ascunse (Fără Neliniaritate)

- Pentru un MLP cu un singur strat ascuns (Figura 4), datele de intrare $\mathbf{X} \in \mathbb{R}^{n \times d}$ sunt transformate în:
  - **Stratul Ascuns (H):** $H \in \mathbb{R}^{n \times h}$, unde $h$ este numărul de unități ascunse:

    $$H = XW^{(1)} + b^{(1)}$$

    unde $W^{(1)} \in \mathbb{R}^{d \times h}$ și $b^{(1)} \in \mathbb{R}^{1 \times h}$ sunt ponderile și bias-urile stratului ascuns.
  - **Stratul de ieșire (O):** $O \in \mathbb{R}^{n \times q}$, unde $q$ este numărul de ieșiri:

    $$O = HW^{(2)} + b^{(2)}$$

    unde $W^{(2)} \in \mathbb{R}^{h \times q}$ și $b^{(2)} \in \mathbb{R}^{1 \times q}$ sunt ponderile și bias-urile stratului de ieșire.

## Limitarea Liniarității (Motivația pentru Neliniaritate)

- Dacă un MLP constă doar din straturi afine stivuite (fără funcție de activare), se poate demonstra formal că el nu oferă niciun beneficiu:
  - O transformare afină a unei transformări afine este **ea însăși o transformare afină**.
  - Modelul liniar (Softmax/Regresie Liniară) era deja capabil să reprezinte orice transformare afină.
- **Echivalența formală:** Fără funcții de activare, stratul ascuns poate fi eliminat, rezultând un model echivalent cu un singur strat (liniar) cu parametrii $\mathbf{W} = \mathbf{W}^{(1)}\mathbf{W}^{(2)}$ și $\mathbf{b} = \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$.

## Ingredientul Cheie: Funcția de Activare Neliniară

- Pentru a realiza arhitecturi multistrat, este necesar un **ingredient cheie**: o **funcție de activare neliniară** $\sigma$.
- Această funcție este aplicată fiecărei unități ascunse imediat după transformarea afină. Ieșirile funcțiilor de activare se numesc **activări (*activations*)**.
- **MLP cu Neliniaritate:**

$$H = \sigma(XW^{(1)} + b^{(1)})$$

$$O = HW^{(2)} + b^{(2)}$$

- **Proprietăți MLP-uri cu neliniarități:** MLP-urile pot capta **interacțiuni complexe** între intrări și, cu anumite funcții de activare, sunt **aproximatori universali (*universal approximators*)**.

## Funcții de Activare Comune

Funcțiile de activare decid dacă un neuron trebuie activat sau nu și transformă semnalele de intrare ponderate în ieșiri.

1. **Rectified Linear Unit (ReLU):**
   - Cea mai populară alegere, simplă și eficientă.
   - **Formula:** $\mathrm{ReLU}(x) = \max(x, 0)$.
   - Efect: Reține doar valorile pozitive și setează la 0 toate valorile negative.
   - Derivata: Este 0 când inputul este negativ și 1 când este pozitiv.
   - Variație: **Parameterized ReLU (pReLU)** adaugă un termen liniar mic $(\alpha \min(0, x))$ pentru inputurile negative.
2. **Sigmoid:**
   - Transformă inputurile din $\mathbb{R}$ în intervalul. Este adesea numită funcție de "squashing".
   - Formula: $\mathrm{sigmoid}(x) = \frac{1}{1+\exp(-x)}$.
   - **Utilizare:** Încă folosită pe straturile de ieșire pentru probleme de **clasificare binară** (pentru a interpreta ieșirile ca probabilități).
   - **Limitare:** A fost înlocuită de ReLU în straturile ascunse, deoarece suferă de problema **gradientului evanescent (*vanishing gradient*)**.
3. **Tanh (Tangentă Hiperbolică):**
   - De asemenea, o funcție de *squashing*, dar mapează inputurile în intervalul **[-1, 1]**.
   - Derivata: Atinge un maxim de 1 când inputul este 0 și se apropie de 0 pe măsură ce inputul se îndepărtează de 0.
   - Ca și Sigmoid, tinde să se comporte ca o transformare liniară când inputul este aproape de 0.

---

# 3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

## Concept Dificil: Eșecul Liniarității (De ce avem nevoie de $\sigma$?)

- **Analogie Simplificată (Buget și Taxe):** Să presupunem că un strat aplică o taxă de 10% și un cost fix de 5 lei (transformare afină 1). Al doilea strat aplică o altă taxă de 5% și un cost fix de 3 lei (transformare afină 2).
  - Indiferent cât de multe astfel de straturi adăugați, rezultatul final poate fi întotdeauna exprimat ca o singură taxă unică și un singur cost fix total.
  - **Fără neliniaritate**, adăugarea de straturi suplimentare este ca și cum ați avea $N$ straturi de sticlă transparentă; produsul final este tot transparent, oferind **zero putere expresivă suplimentară**.

## Termen Tehnic: Universal Approximators

- **Clarificare:** Această proprietate nu înseamnă că un MLP *poate învăța* orice funcție perfect în practică. Înseamnă că, **teoretic**, dat fiind un număr suficient de noduri (posibil absurd de multe) și ponderi corecte, o rețea MLP cu un singur strat ascuns poate modela *orice funcție*.
- *Asta se întâmplă deoarece:* Adăugarea neliniarităților (precum ReLU) permite MLP-urilor să creeze regiuni de decizie complexe și să combine interacțiuni neliniare ale intrărilor, capturând relații pe care un model liniar nu le-ar putea vedea.

### Funcția de Activare ReLU

- **Avantaj: Gradienți Buni:** Motivul popularității ReLU este că derivatele sale sunt "bine comportate". Derivata este fie 0 (pentru input negativ), fie 1 (pentru input pozitiv).
- *Asta se întâmplă deoarece:* Acest lucru ajută la atenuarea problemei documentate a **gradientului evanescent** (vanishing gradient) care a afectat versiunile anterioare de rețele care foloseau Sigmoid sau tanh, făcând optimizarea mult mai ușoară.

---

## 4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (MLP și Complexitate):**
  - **CAUZĂ:** Simpla stivuire a straturilor liniare/afine nu aduce beneficii.
  - **EFECT:** Necesitatea absolută de a introduce **neliniaritatea** $\sigma$ (funcții de activare) între straturile afine pentru ca MLP-ul să poată capta interacțiuni complexe și să devină un aproximator universal.
- **Vocația Stratului de Ieșire:** Ultimele $L-1$ straturi (cele ascunse) sunt folosite pentru a genera o **reprezentare utilă** a datelor. Stratul final (output layer) rămâne întotdeauna un **predictor liniar** care folosește acea reprezentare pentru a face predicția finală (regresie liniară sau logistic/Softmax).
- **Avantajul Modern (Context Istoric):** Deși cunoștințele noastre ne plasează la nivelul unui practician din jurul anului 1990, astăzi avem un avantaj imens: putem folosi framework-uri moderne de *deep learning* (open-source) pentru a construi rapid aceste modele, în loc să scriem mii de linii de cod C și Fortran.

---

## 5. EXERCIȚII ȘI REZOLVARE

Vom folosi notația matricială și modelul de exercițiu din sursa "02_09_Exercises.pdf".

### Exercițiul M.1: Calculul Ieșirii (Forward Pass) și Costul Parametrilor pentru MLP

Definiți un MLP pentru regresie, având $d = 3$ intrări, un singur strat ascuns cu $h = 2$ unități, folosind funcția de activare $\mathrm{ReLU}(x) = \max(x, 0)$, și $q = 1$ ieșire.

Pentru input-ul $X = \begin{bmatrix} 1 & 0 & 1 \\ -1 & 1 & 0 \end{bmatrix}$ (deci $n = 2$ exemple).

Folosim următoarele setări de parametri (similar cu):

$$W^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix} \in \mathbb{R}^{3\times 2}, \quad b^{(1)} = \begin{bmatrix} 1 & 0 \end{bmatrix} \in \mathbb{R}^{1\times 2}$$

$$W^{(2)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \in \mathbb{R}^{2\times 1}, \quad b^{(2)} = [1] \in \mathbb{R}^{1\times 1}$$

1. Calculați ieșirea $\mathbf{O}$ a rețelei (procesul *forward pass*).
2. Specificați numărul total de parametri (greutăți și bias-uri) ai rețelei.

**Rezolvare M.1:**

**Formule Necesare:**

$$H = \mathrm{ReLU}(XW^{(1)} + b^{(1)})$$

$$O = HW^{(2)} + b^{(2)}$$

$$\mathrm{ReLU}(x) = \max(x, 0)$$

**1. Calculul Ieșirii O:**

**Pasul 1.1: Calculul scorurilor liniare pentru Stratul Ascuns $XW^{(1)}$:**

$$XW^{(1)} = \begin{bmatrix} 1 & 0 & 1 \\ -1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} (1\cdot 1 + 0\cdot 0 + 1\cdot 1) & (1\cdot 0 + 0\cdot 1 + 1\cdot(-1)) \\ (-1\cdot 1 + 1\cdot 0 + 0\cdot 1) & (-1\cdot 0 + 1\cdot 1 + 0\cdot(-1)) \end{bmatrix} = \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix}$$

**Pasul 1.2: Adăugarea Bias-ului $b^{(1)}$:**

$$XW^{(1)} + b^{(1)} = \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 3 & -1 \\ 0 & 1 \end{bmatrix}$$

*Notă: Bias-ul $b^{(1)} = [1\ 0]$ este extins (broadcasted) la dimensiunea $2 \times 2$ pentru a fi adunat.*

**Pasul 1.3: Aplicarea Funcției de Activare ReLU:**

$$H = \mathrm{ReLU}\left(\begin{bmatrix} 3 & -1 \\ 0 & 1 \end{bmatrix}\right) = \begin{bmatrix} \max(3,0) & \max(-1,0) \\ \max(0,0) & \max(1,0) \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \in \mathbb{R}^{2\times 2}$$

**Pasul 1.4: Calculul ieșirii O:**

$$O = HW^{(2)} + b^{(2)} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3\cdot 1 + 0\cdot 0 \\ 0\cdot 1 + 1\cdot 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$O = \begin{bmatrix} 3 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \end{bmatrix} \in \mathbb{R}^{2\times 1}$$

Ieșirea finală a rețelei este $\mathbf{O} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}$.

**2. Numărul Total de Parametri:**

- Ponderi $W^{(1)}$: $d \times h = 3 \times 2 = 6$.
- Bias-uri $b^{(1)}$: $h = 2$.
- Ponderi $W^{(2)}$: $h \times q = 2 \times 1 = 2$.
- Bias-uri $b^{(2)}$: $q = 1$.
- Total parametri: $6 + 2 + 2 + 1 = \mathbf{11}$.

## Exercițiul M.2: Rolul Neliniarității

Considerați un MLP cu două straturi, dar în care funcția de activare $\sigma$ este omisă (adică $\sigma(x) = x$, transformarea identitate).

1. Explicați de ce acest model nu este mai puternic din punct de vedere expresiv decât o Regresie Liniară simplă.
2. De ce funcțiile Sigmoid și tanh (dar nu și ReLU) au fost înlocuite în straturile ascunse ale MLP-urilor moderne?

**Rezolvare M.2:**

1. **Lipsa Puterii Expresive:** Fără o funcție de activare neliniară, MLP-ul este o compoziție de transformări afine (liniare plus bias). O compunere de funcții afine este întotdeauna **o singură funcție afină echivalentă**. Prin urmare, chiar dacă antrenăm o rețea cu mai multe straturi, aceasta poate reprezenta doar o dreaptă (sau un hiperplan) în spațiul de intrare, la fel ca un model liniar cu un singur strat, adică **nu se obține nimic în plus**.
2. **Înlocuirea Sigmoid și tanh:** Sigmoid și tanh au fost înlocuite în straturile ascunse de către ReLU, deoarece:
   - Sufereau de problema bine documentată a **gradientului evanescent (*vanishing gradient*)**. Derivatele lor se apropie de zero când inputul este mare sau mic.
   - ReLU are derivate bine comportate (0 sau 1), ceea ce facilitează optimizarea și ameliorează problema *vanishing gradient*.

# 2.3

Bună ziua! Mă bucur să ne concentrăm pe această lecție crucială. Secțiunea 2.3 ne mută de la *cum funcționează* un model la *cât de bine* funcționează acesta în lumea reală. Vom explora problema fundamentală a Machine Learning: **generalizarea**.

---

# Ghid de Studiu Complet: 2.3 Selecția Modelului, Underfitting și Overfitting

## 1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Această lecție abordează cea mai importantă problemă fundamentală a Machine Learning-ului: **generalizarea**. Scopul în ML este de a descoperi tipare (*patterns*) care captează regularități în populația subiacentă din care au fost extrase datele noastre de antrenare. Succesul se măsoară prin capacitatea modelului de a evalua corect performanța pe **eșantioane pe care nu le-a întâlnit niciodată înainte**. Deoarece antrenăm modele pe eșantioane mici și finite, există un risc inerent de a descoperi asocieri aparente care nu se susțin când colectăm mai multe date. Acest risc se manifestă sub formă de **underfitting** (model prea simplu) sau **overfitting** (model prea complex, care a memorat zgomotul). Pentru a combate aceste probleme și a alege modelul optim, trebuie să diferențiem între eroarea de antrenare și eroarea de generalizare și să folosim un set de date independent numit **set de validare (*validation dataset*)**.

**Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?**

1. **Generalizarea este Obiectivul:** Obiectivul fundamental al ML este de a descoperi tipare care generalizează, nu doar de a memora datele de antrenare.
2. **Overfitting (Supra-antrenare):** Fenomenul de potrivire a datelor de antrenare mai fidel decât distribuția subiacentă. Este combătut prin **tehnici de regularizare**.
3. **Underfitting (Sub-antrenare):** Modelul este insuficient de expresiv (prea simplu) pentru a capta tiparul real.
4. **Estimarea Erorii: Eroarea de antrenare** este calculată pe setul de antrenare, iar **Eroarea de generalizare** este estimată folosind un **set de testare independent** (sau un set de validare pentru selecția modelului).
5. **Selecția Modelului (Model Selection):** Procesul de alegere a hiperparametrilor optimi (complexitate optimă) se face exclusiv prin evaluarea performanței pe un **set de validare**, pentru a evita supra-antrenarea datelor de testare.

---

# 2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

## 2.3 Selecția Modelului, Underfitting și Overfitting

**Problema Generalizării și Datele Limitate**

- În ML, obiectivul este de a descoperi tipare.
- Când antrenăm modele, accesăm doar un eșantion mic de date.
- Cele mai mari seturi de date de imagini publice conțin aproximativ un milion de imagini, dar de multe ori lucrăm cu mii sau zeci de mii de exemple.
- Lucrul cu eșantioane finite (finite samples) riscă să ducă la descoperirea de asocieri aparente care nu se mențin atunci când se colectează mai multe date.

**Definițiile Erorilor**

- **Eroarea de antrenare (*training error*)** este eroarea modelului calculată pe setul de date de antrenare.
- **Eroarea de generalizare (*generalization error*)** este eroarea așteptată a modelului dacă ar fi aplicat unui flux infinit de exemple suplimentare extrase din aceeași distribuție de date subiacentă.
- Deoarece nu putem calcula exact eroarea de generalizare (fluxul infinit este un obiect imaginar), în practică, trebuie să o **estimăm** aplicând modelul pe un **set de testare independent** (independent test set).
- Se presupune **ipoteza i.i.d. (*i.i.d. assumption*)** (independently from identical distributions): atât datele de antrenare, cât și cele de testare sunt extrase independent din distribuții identice.

**Overfitting și Underfitting (Relația cu Complexitatea Modelului)**

- **Overfitting:** Fenomenul de potrivire a datelor de antrenare mai fidel decât distribuția subiacentă.
  - Apare când funcția este atât de flexibilă încât poate capta tipare false (spurious patterns) la fel de ușor ca și asocierile adevărate.
  - Tehnicile folosite pentru a combate overfitting-ul sunt **tehnicile de regularizare**.
  - Se indică prin **eroarea de antrenare semnificativ mai mică** decât eroarea de validare, arătând un *severe overfitting*.
- **Underfitting:** Se observă când eroarea de antrenare și eroarea de validare sunt ambele **substanțiale**, cu un **ecart mic (*little gap*)** între ele.
  - Indică faptul că modelul este **prea simplu** (insufficiently expressive) pentru a capta tiparul.
  - Se recomandă utilizarea unui model mai complex.

**Factori care Influențează Generalizabilitatea (Punctele 1, 2, 3)**

1. **Numărul de parametri reglabili (*tunable parameters*)**, numiți uneori **grade de libertate (*degrees of freedom*)**: Când acest număr este mare, modelele sunt mai susceptibile la overfitting.
2. **Valorile luate de parametri:** Când ponderile pot lua o gamă mai largă de valori, modelele sunt mai susceptibile la overfitting.
3. **Numărul de exemple de antrenare:** Este trivial de ușor să supra-antrenezi un set de date cu doar unul sau două exemple. Suprafața-antrenarea unui set de date cu milioane de exemple necesită un model extrem de flexibil.

**Mărimea Setului de Date (*Dataset size*)**

- Pentru un model fix, un număr mai mic de eșantioane în setul de antrenare crește probabilitatea (și severitatea) overfitting-ului.
- În general, pe măsură ce creștem cantitatea de date de antrenare, eroarea de generalizare scade. Mai multe date nu strică niciodată (*more data never hurts*).

- Succesul actual al *deep learning*-ului se datorează, parțial, abundenței actuale de seturi de date masive.

**Selecția Modelului (Model Selection)**

- Selecția modelului este procesul de alegere a modelului final prin evaluarea mai multor modele candidate.
- Compararea se poate face între clase de modele fundamental diferite (ex: arbori de decizie vs. rețele neuronale) sau între membri ai aceleiași clase cu **setări diferite de hiperparametri** (ex: numărul de straturi ascunse într-un MLP).
- Pentru a determina cel mai bun model, se folosește un **set de validare (*validation dataset*)**.
- **Atenție:** Nu trebuie atins setul de testare (test set) până când toți hiperparametrii nu au fost aleși. Utilizarea datelor de testare în procesul de selecție a modelului riscă să **supra-antreneze datele de testare** (*overfit the test data*), ceea ce este problematic.
- **Practica Comună:** Datele sunt împărțite în trei seturi: antrenare (*training*), validare (*validation*), și testare (*test*).

**K-fold Cross-Validation**

- Când datele de antrenare sunt insuficiente pentru a constitui un set de validare adecvat, se folosește **validarea încrucișată K-fold (*K-fold cross-validation*)**.
- Datele de antrenare originale sunt împărțite în  subseturi non-suprapuse.
- Modelul este antrenat și validat de  ori, folosind $-1$ subseturi pentru antrenare și un subset diferit pentru validare în fiecare rundă.
- Erorile de antrenare și validare sunt calculate făcând media rezultatelor celor experimente.

**Principiul Lamei lui Occam (Occam's razor)**

- Conform principiului Lamei lui Occam, dat fiind două explicații care se potrivesc bine datelor, **cea mai simplă** (cea care face mai puține presupuneri) este cel mai probabil să fie corectă.
- Un model care poate explica ușor fapte arbitrare este considerat complex, în timp ce un model care are putere expresivă limitată, dar totuși reușește să explice bine datele, este mai aproape de adevăr.

---

# 3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

## Concept Dificil: Ipoteza i.i.d. (Independently from Identical Distributions)

- **Explicație Simplificată:** Această ipoteză este baza matematică a ML-ului supervizat.
  - **"Distribuții Identice":** Toate datele (de antrenare și testare) provin din aceeași "sursă" sau regulă. (Ex: Dacă antrenez un model să recunoască pisici, toate imaginile de pisici trebuie să respecte aceleași legi fizice de bază, chiar dacă pisicile sunt diferite.)
  - **"Independent":** Fiecare eșantion este extras fără nicio relație cu eșantioanele anterioare.
- *Asta se întâmplă deoarece:* Această presupunere asigură că setul nostru de testare (care estimează generalizarea) este o reprezentare fidelă și necontaminată a distribuției din care a fost extras setul de antrenare.

### Vizualizarea Underfitting vs. Overfitting (Conform Figurii 11)

- **Analogie:** Gândiți-vă la un croitor care încearcă să facă un costum (modelul) pentru un client (distribuția subiacentă) folosind doar o mostră de măsurători (setul de antrenare). | Regim | Complexitate Model | Eroare Antrenare (Homework) | Eroare Generalizare (Examen) | Problema | | :--- | :--- | :--- | :--- | :--- | | **Underfitting** | Foarte scăzută | Substanțială | Substanțială (Gap mic) | Croitorul folosește un tipar standard, nepotrivit pentru client. | | **Optimum** | Medie | Mică | Minimă | Croitorul face ajustări perfecte pentru mărimea reală. | | **Overfitting** | Foarte ridicată | Foarte mică (aproape 0) | Mare (Gap mare) | Croitorul a reprodus și cutele și micile pete de pe mostra de măsurători (zgomot). |
- *Asta se întâmplă deoarece:* Creșterea complexității reduce eroarea de antrenare (curba de jos), dar la un moment dat începe să crească eroarea de generalizare (curba de sus). Punctul optim este locul în care cele două curbe se separă cel mai puțin, minimizând totodată curba de generalizare.

---

## 4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Flexibilitate și Overfitting):**
  - **CAUZĂ:** Un model are prea multe grade de libertate (număr mare de parametri reglabili) și/sau poate lua o gamă largă de valori.
  - **EFECT:** Modelul devine prea flexibil și începe să se potrivească zgomotului, nu doar tiparului real.
  - **SOLUȚIE:** Tehnici de regularizare (precum cele discutate în Lecția 1.5, Ridge și Lasso) sunt folosite pentru a controla sau a reduce această flexibilitate excesivă.
- **Importanța Validării:** Deoarece nu ne putem baza pe eroarea de antrenare (care scade mereu odată cu creșterea complexității modelului) și nu ar trebui să atingem setul de testare, **setul de validare** devine singurul instrument onest pentru a alege între modele.
- **MLP și Generalizare (Conexiunea cu Lecția 2.2):** Am învățat că MLP-urile sunt aproximatori universali, adică sunt foarte expresivi și flexibili. Deși acest lucru este

benefic (pentru a combate *underfitting*-ul), flexibilitatea lor inerentă (capacitatea de a explica fapte arbitrare) îi face deosebit de susceptibili la **overfitting**, necesitând utilizarea riguroasă a tehnicilor de regularizare și a seturilor de validare.

---

# 5. EXERCIȚII ȘI REZOLVARE

Vom genera exerciții conceptuale bazate pe logica de selecție a modelului, urmând structura din sursa.

## Exercițiul S.1: Evaluarea Underfitting-ului și Overfitting-ului

O echipă antrenează un model MLP și înregistrează următoarele erori pentru trei arhitecturi diferite pe un set de date de antrenare și pe un set de validare independent.

| Arhitectură | Complexitate (Grade de Libertate) | E. Antrenare (Loss) | E. Validare (Loss) |
|---|---|---|---|
| Model A | Mică | 25.0 | 26.0 |
| Model B | Medie | 8.5 | 9.0 |
| Model C | Mare | 1.0 | 15.0 |

1. Identificați modelul care suferă de **underfitting**. Motivați răspunsul, menționând tipul de *gap* (ecart).
2. Identificați modelul care suferă de **overfitting sever**. Motivați răspunsul.
3. Care model ar trebui ales (Optimum) și ce sugerează acest lucru despre complexitatea optimă pentru problema dată?

**Rezolvare S.1:**

1. **Underfitting (Model A):** Modelul A are erori mari (25.0 și 26.0) pe ambele seturi, dar **ecartul (*gap*) dintre ele este mic** (1.0). Acest lucru indică faptul că modelul este **prea simplu** (insufficiently expressive) pentru a reduce eroarea de antrenare.
2. **Overfitting Sever (Model C):** Modelul C are o eroare de antrenare foarte mică (1.0), dar o eroare de validare extrem de mare (15.0). Ecartul semnificativ (14.0) indică faptul că modelul s-a potrivit mult prea strâns datelor de antrenare, capturând zgomotul.
3. **Modelul Optim (Model B):** Modelul B oferă cea mai mică eroare de generalizare (9.0). Deși eroarea de antrenare nu este cea mai mică (8.5), diferența față de eroarea de validare este minimă, sugerând cel mai bun echilibru de generalizare (Optimum).

## Exercițiul S.2: Rolul Setului de Testare (Test Set)

De ce se folosește un set de validare independent pentru selecția hiperparametrilor (Model Selection) în locul setului de testare?

**Rezolvare S.2:**

1. **Setul de Testare este Sacrosanct:** Conform principiului selecției modelului, setul de testare **nu trebuie atins** până când toți hiperparametrii nu au fost aleși.
2. **Riscul de Contaminare:** Dacă am folosi setul de testare pentru a regla hiperparametrii (cum ar fi rata de învățare sau gradul polinomial), există riscul de a **supra-antrena datele de testare** (*overfit the test data*).
3. **Măsura Finală:** Setul de testare trebuie să rămână o măsură *onestă* a performanței modelului pe date nevăzute. Dacă supra-antrenăm setul de testare, nu vom ști niciodată cât de bine se va generaliza modelul în lumea reală.

# 2.4

Salutări! Astăzi vom aprofunda un instrument esențial în lupta împotriva supra-antrenării (overfitting): **Weight Decay** (Decăderea Ponderilor).

---

# Ghid de Studiu Complet: 2.4 Decăderea Ponderilor (Weight Decay)

## 1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Această lecție introduce **Decăderea Ponderilor (Weight Decay)**, o tehnică standard, cunoscută în general sub numele de **regularizare** $\ell_2$, utilizată pe scară largă pentru regularizarea modelelor parametrice de *machine learning*. După ce am caracterizat problema *overfitting*-ului (Lecția 2.3), ne concentrăm pe soluții practice. Deși colectarea de date suplimentare atenuează întotdeauna *overfitting*-ul, acest lucru este adesea costisitor sau imposibil pe termen scurt. Soluția *Weight Decay* este motivată de intuiția de bază, conform căreia **cele mai simple modele** sunt cele pentru care o normă a vectorului de ponderi este **cât mai mică posibil**. Metoda realizează acest lucru prin **adăugarea unui termen de penalizare** (bazat pe norma $\ell_2$ a ponderilor) la funcția de pierdere originală.

**Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?**

1. **Definiție și Obiectiv:** *Weight decay* (regularizarea $\ell_2$) este cea mai utilizată tehnică pentru a combate *overfitting*-ul prin **menținerea ponderilor modelului cât mai mici posibil**.
2. **Mecanism:** Se înlocuiește obiectivul inițial (minimiza pierderea predicției) cu un nou obiectiv: **minimiza suma pierderii predicției și a termenului de penalizare**.
3. **Formula (Norma Frobenius):** Penalizarea utilizează **norma Frobenius** ($||\mathbf{W}||_F^2$), care este o generalizare a normei Euclidiene $\ell_2$ pentru matrice (în MLP).
4. **Efect în Optimizare:** Atunci când se aplică Coborârea Gradientului, această tehnică **micșorează (decays) mărimea ponderilor** către zero la fiecare pas de antrenare, pe lângă actualizarea bazată pe eroare.

# 2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

## 2.4 Decăderea Ponderilor (Weight Decay)

- **Context:** Deși *overfitting*-ul poate fi atenuat prin colectarea de date de antrenare suplimentare, acest lucru poate fi costisitor, consumator de timp sau imposibil pe termen scurt. Prin urmare, ne concentrăm pe tehnicile de regularizare.
- **Motivație:** Intuiția de bază este că modelele **cele mai simple** sunt cele care au o normă a vectorului de ponderi **cât mai mică posibil**.
- **Obiectivul Modificat:** Obiectivul original de a minimiza pierderea predicției pe etichetele de antrenare este înlocuit cu un obiectiv nou: **minimiza suma pierderii predicției și a termenului de penalizare**.
- Dacă vectorul de ponderi crește prea mult, algoritmul de învățare se va concentra pe minimizarea normei ponderilor, versus minimizarea erorii de antrenare.

## Formularea Matematică (Exemplu MLP)

- Termenul de penalizare se adaugă la funcția de pierdere.
- **Funcția de Pierdere Regulatizată** $\mathcal{L}(\mathbf{w})$ pentru un MLP cu un singur strat ascuns (folosind MSE ca pierdere de predicție) devine:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2 + \frac{\lambda}{2}\left(||\mathbf{W}^{(1)}||_F^2 + ||\mathbf{W}^{(2)}||_F^2\right)$$

- **Norma Frobenius:** $||\mathbf{W}||_F$ este **norma Frobenius**, care este o generalizare a normei Euclidiene $\ell_2$ pentru matrice.
  - Este definită ca: $||\mathbf{W}||_F = \sqrt{\mathrm{Tr}(\mathbf{W}^\top\mathbf{W})}$.
- **Constanta de Regularizare ($\lambda$):** $\lambda$ este un **hiperparametru nengativ** (nonnegative hyperparameter) care caracterizează compromisul dintre pierderea standard și penalizarea aditivă.
  - $\lambda$ este ajustat (fit) folosind **date de validare**.
  - Valori mai mici ale lui $\lambda$ constrâng ponderile mai puțin, în timp ce valori mai mari ale lui $\lambda$ constrâng ponderile mai mult.

## Considerații Computaționale

- **Cazul $\lambda = 0$:** Se recuperează funcția de pierdere originală, fără regularizare.
- **Cazul $\lambda > 0$:** Mărimea ponderilor rețelei este restricționată.
- **Utilizarea Normei Pătrate ($\ell_2^2$):** Se folosește norma la pătrat ($||\mathbf{W}||_F^2$) și se împarte la 2 din **considerente de convenienţă computaţională**.
  - Atunci când se calculează derivata unei funcții pătratice, factorii 2 și 1/2 se anulează, simplificând expresia pentru actualizare.

- Prin ridicarea la pătrat a normei $\ell_2$, se elimină rădăcina pătrată, rămânând doar suma pătratelor fiecărei componente a vectorului/matricei de ponderi.
- Acest lucru face ca derivata termenului de penalizare să fie ușor de calculat: derivata sumei este egală cu suma derivatelor.

### Efectul Asupra Antrenării (Mini-Batch SGD)

- Când se aplică algoritmul Mini-Batch Stochastic Gradient Descent (SGD), pe lângă actualizarea ponderilor bazată pe diferența dintre predicție și eticheta reală, metoda, de asemenea, **micșorează mărimea ponderilor $\mathbf{W}^{(1)}$ și $\mathbf{W}^{(2)}$ către zero**.
- Acest efect de micșorare este motivul pentru care metoda este numită uneori **"weight decay"**.

---

## 3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

### Termen Tehnic: Norma Frobenius ($||\mathbf{W}||_F$)

- **Explicație Simplificată (ELI5):** Norma Frobenius este doar o modalitate elegantă de a măsura "cât de mare" este o matrice.
  - Dacă o matrice ar fi un vector lung, norma Frobenius ar fi exact lungimea acelui vector în spațiul Euclidian ($\ell_2$).
  - **Scop:** Folosind această normă, penalizăm mărimea absolută a tuturor ponderilor din acel strat (matricea $\mathbf{W}$), forțându-le să se apropie de zero colectiv.

### Concept Dificil: Convenienței Computațională a Pătratului Normei

- **Context:** De ce nu folosim norma standard $\ell_2$, $\sum |w_i|$?
- *Asta se întâmplă deoarece:* Funcțiile de pierdere trebuie să fie ușor de derivat pentru a folosi Gradient Descent.
  - Dacă funcția de penalizare este **pătratică** (adică $w^2$), derivata este simplă ($2w$). Prin includerea factorului de $\frac{1}{2}$ în fața termenului de penalizare, derivata devine exact $w$.
  - Dacă am folosi rădăcina pătrată (norma $\ell_2$), derivarea ar fi mult mai complexă. Folosirea normei pătrate face ca expresia de actualizare să arate **"frumos și simplu"**.

### Funcția lui $\lambda$ (Trade-off)

- **Analogie (Pedala de Frână):** Hiperparametrul $\lambda$ acționează ca o pedală de frână pentru model.

- Minimizarea erorii de antrenare este ca și cum modelul ar accelera (încercând să potrivească datele cât mai bine).
- Penalizarea $\ell_2$ este frâna.
- $\lambda$ controlează cât de puternic apăsați pe frână. Dacă $\lambda$ este mare, frâna este puternică, iar ponderile sunt puternic trase înapoi spre zero, prevenind *overfitting*-ul, dar riscând *underfitting*-ul. Dacă $\lambda$ este zero, frâna este eliberată.

---

## 4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Conexiunea cu Overfitting (Lecția 2.3):** Această tehnică este o metodă directă de regularizare, necesară odată ce problema *overfitting*-ului a fost caracterizată. Ea combate *overfitting*-ul prin reducerea numărului efectiv de grade de libertate ale modelului, forțându-l să prefere soluțiile mai simple.
- **Conexiunea cu Regularizarea Liniară (Lecția 1.5):** *Weight decay* în MLP este exact același principiu ca **Ridge Regression** pentru modelele liniare simple. Ambele folosesc penalizarea normei $\ell_2$. Singura diferență este că, pentru MLP, se folosește norma Frobenius (care este norma $\ell_2$ generalizată pentru matrice).
- **Selecția Hiperparametrului:** Așa cum am stabilit în Lecția 2.3, hiperparametrul $\lambda$ nu poate fi reglat pe setul de antrenare. Trebuie ajustat prin **evaluarea performanței pe un set de validare**.

---

## 5. EXERCIȚII ȘI REZOLVARE

Vom folosi structura de formule MLP și notațiile de matrice din surse.

### Exercițiul D.3: Identificarea Componentelor Funcției de Cost Regularizate

Se consideră un MLP cu două straturi, antrenat pe o problemă de regresie. Funcția de pierdere regularizată (Weight Decay) este dată de:

$$\mathcal{L}(\mathbf{w}) = \mathcal{L}_{\text{original}}(\mathbf{w}) + \frac{\lambda}{2}\left(||\mathbf{W}^{(1)}||_F^2 + ||\mathbf{W}^{(2)}||_F^2\right)$$

1. Ce reprezintă termenul $\frac{\lambda}{2}\left(||\mathbf{W}^{(1)}||_F^2 + ||\mathbf{W}^{(2)}||_F^2\right)$ și cum este numit?
2. Ce reprezintă $\lambda$ și de ce este important să fie ajustat folosind date de validare?
3. Ce se întâmplă cu obiectivul de optimizare dacă matricea de ponderi $\mathbf{W}^{(1)}$ devine foarte mare?

**Rezolvare D.3:**

1. Termenul reprezintă **termenul de penalizare**, care este bazat pe suma pătratelor normelor Frobenius ale tuturor matricilor de ponderi din rețea. Este asociat metodei **Weight Decay** sau **regularizarea** $\ell_2$.
2. $\lambda$ este **constanta de regularizare**, un hiperparametru nengativ. Este important să fie ajustat pe date de validare deoarece caracterizează **compromisul** dintre minimizarea erorii de antrenare și minimizarea normei ponderilor. Utilizarea setului de testare ar risca supra-antrenarea acestuia (conform Lecției 2.3).
3. Dacă $\mathbf{W}^{(1)}$ devine foarte mare, termenul de penalizare devine dominant, iar algoritmul de învățare se va concentra pe **minimizarea normei ponderilor** (reducerea $\mathbf{W}^{(1)}$) mai degrabă decât pe reducerea erorii de antrenare.

### Exercițiul D.4: Gradient Descent și Weight Decay

1. Explicați de ce metoda este numită "weight decay" atunci când este utilizată cu Mini-Batch Stochastic Gradient Descent (SGD).
2. Care este motivul matematic al convenienței pentru utilizarea $\ell_2^2$ în locul $\ell_2$ în funcția de pierdere regularizată?

**Rezolvare D.4:**

1. Metoda este numită "weight decay" deoarece, atunci când se aplică algoritmul Mini-Batch SGD, pe lângă actualizarea ponderilor bazată pe diferența dintre predicție și etichetă, procesul **micșorează mărimea ponderilor** $\mathbf{W}^{(1)}$ **și** $\mathbf{W}^{(2)}$ **către zero** la fiecare pas de antrenare. Penalizarea $\ell_2$ în sine forțează această "decădere".
2. Utilizarea **normei pătrate** (ex: $||\mathbf{W}||_F^2$) este preferată pentru **convenienței computațională**. Prin folosirea normei pătrate:
   - Rădăcina pătrată este eliminată, lăsând suma pătratelor fiecărei componente.
   - Prin convenție, termenul este împărțit la 2. Când se calculează derivata (necesară pentru GD), 2 și 1/2 se anulează, simplificând semnificativ expresia de actualizare.

## 2.5

Bun găsit! Ne concentrăm astăzi pe o tehnică de regularizare extrem de eficientă, care a revoluționat antrenarea rețelelor neuronale profunde: **Dropout**. Această lecție contrastează *Dropout* cu regularizarea clasică ($\ell_2$ Weight Decay) și explică mecanismul său inteligent de a preveni co-adaptarea neuronilor.

---

# Ghid de Studiu Complet: 2.5 Dropout

## 1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 2.5 **Dropout** continuă discuția despre tehnicile de regularizare, arătând că, deși regularizarea $\ell_2$ (Weight Decay) este o metodă clasică și eficientă, rețelele neuronale

profunde (deep neural networks) au o flexibilitate și expresivitate atât de mare încât pot supra-antrena (overfit) chiar și atunci când există mult mai multe exemple decât caracteristici. **Dropout** este o tehnică standard de antrenare a rețelelor neuronale care abordează direct această problemă prin **injectarea de zgomot (injecting noise)** în straturile interne ale rețelei în timpul propagării înainte (*forward propagation*). Metoda funcționează prin **dezactivarea aleatorie a unui număr de neuroni (drop out some neurons)** la fiecare iterație, prevenind astfel **co-adaptarea** și forțând modelul să răspândească ponderile (*spread out its weights*) pe mai multe caracteristici, în loc să se bazeze excesiv pe asocieri potențial false.

**Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?**

1. **Obiectivul (Simplitate și Finețe):** Dropout, ca și $\ell_2$, urmărește să obțină un model **simplu**, dar introduce o noțiune suplimentară de simplitate: **finețea (*smoothness*)**, adică funcția nu ar trebui să fie sensibilă la zgomot.
2. **Mecanismul de Antrenare:** În timpul antrenării, *dropout* implică **setarea la zero** a unei fracțiuni de noduri (neuroni) din fiecare strat ascuns (intermediate activation) cu o **probabilitate** $p$.
3. **Co-adaptarea:** Dropout combate *overfitting*-ul prin ruperea **co-adaptării** (co-adaptation), o stare în care un strat depinde excesiv de un tipar specific de activări din stratul anterior.
4. **Normalizarea:** Ieșirea straturilor (*layer activation*) este **normalizată** prin împărțirea la $(1-p)$ (fracția de noduri reținute), astfel încât valoarea așteptată $E[h']$ să rămână neschimbată.
5. **Test Time: Dropout este dezactivat la momentul testării** (*at test time*), iar modelul folosește toți neuronii (și, prin urmare, nu necesită normalizare suplimentară la acel moment).

---

# 2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

## 2.5.1 Contextul Regularizării Liniară

- Abordarea clasică de regularizare a modelelor statistice se face prin penalizarea **normei $\ell_2$ a ponderilor** (Weight Decay).
- În termeni probabilistici, regularizarea $\ell_2$ este justificată prin asumarea unei credințe *a priori* (prior belief) că ponderile iau valori dintr-o **distribuție Gaussiană cu media zero**.
- Intuitiv, $\ell_2$ încurajează modelul să **răspândească ponderile** pe mai multe caracteristici, în loc să depindă prea mult de un număr mic de asocieri potențial false.

## 2.5.2 Limitări ale Modelelor Liniară vs. Rețele Neuronale Profunde

- Modelele liniare suferă de **bias ridicat**, deoarece pot reprezenta doar o clasă restrânsă de funcții, dar au **varianță scăzută** (rezultate similare pe eșantioane diferite).
- Rețelele neuronale profunde ocupă capătul opus al spectrului **bias-variance trade-off**.
  - Spre deosebire de modelele liniare, rețelele neuronale nu se limitează la a privi fiecare caracteristică individual, ci pot **învăța interacțiuni între grupuri de caracteristici**.
  - Acestea sunt atât de expresive încât pot supra-antrena (overfit) chiar și atunci când există **mult mai multe exemple decât caracteristici** (de exemplu, un experiment din 2017 a arătat că rețelele profunde pot eticheta perfect imagini etichetate aleatoriu în setul de antrenare).

### 2.5.3 Principiul Simplității (Smoothness)

- Pentru a reduce decalajul de generalizare dintre performanța pe setul de antrenare și setul de testare, ar trebui să căutăm un model **simplu**.
- Simplitatea poate fi:
  - Un număr mic de dimensiuni.
  - O normă mică a ponderilor (Weight Decay).
- O altă noțiune utilă de simplitate este **finețea (*smoothness*)**: funcția nu ar trebui să fie sensibilă la zgomot. De exemplu, adăugarea de zgomot aleatoriu la pixeli ar trebui să fie inofensivă.

### 2.5.4 Mecanismul Dropout (Injectarea de Zgomot)

- Ideea de bază a *Dropout* este **injectarea de zgomot** în fiecare strat al rețelei în timpul calculului propagării înainte.
- *Dropout* a devenit o tehnică standard pentru antrenarea rețelelor neuronale.
- Metoda se numește *dropout* deoarece **dezactivăm (zeroing out) o fracțiune de neuroni** în fiecare strat la fiecare iterație de antrenare.
- Vizualizare (Figura 12): Se observă cum, după *dropout*, un subset al neuronilor originali (ex: $h_2$ și $h_5$) sunt îndepărtați.

### 2.5.5 Co-adaptarea și Formula de Regularizare

- **Co-adaptarea:** Overfitting-ul în rețelele neuronale este adesea caracterizat de o stare în care fiecare strat se bazează pe un tipar specific de activări din stratul anterior.
- **Dropout** rupe co-adaptarea, nepermițând neuronilor să devină **prea dependenți unul de celălalt**.
- **Formula Standard de Regularizare (Probabilitatea $p$):** În *standard dropout regularization*, activarea intermediară $h$ este înlocuită de o variabilă aleatorie $h'$:

$$h' = \begin{cases} 0, & \text{cu probabilitatea } p \\ \frac{h}{1-p}, & \text{altfel} \end{cases}$$

- Probabilitatea $p$ este **probabilitatea de dezactivare** (*dropout probability*).
- **Normalizare:** Fiecare strat este **normalizat** prin împărțirea la fracția de noduri reținute ($1-p$), astfel încât valoarea așteptată (*expectation*) $E[h']$ să rămână neschimbată, adică $E[h'] = h$.

### 2.5.6 Dropout în Timpul Testării

- **Dropout este dezactivat la momentul testării** (*at test time*).
- Un model antrenat folosește toți neuronii în timpul testării și, din moment ce nu se face dezactivare, nu este necesară nicio normalizare.

---

## 3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

### Termen Tehnic: Co-adaptarea (Co-adaptation)

- **Analogie Simplificată (Echipa de Studiu Frauduloasă):** Imaginați-vă că neuronii sunt membri ai unei echipe care rezolvă o problemă. În overfitting, echipa devine "co-adaptată": Neuronul A așteaptă întotdeauna ca Neuronul B să facă o greșeală specifică, iar Neuronul C corectează acea greșeală, dar numai într-un context specific datelor de antrenare.
- **Cum Rezolvă Dropout:** Prin *dropout*, Neuronul B poate fi eliminat aleatoriu la orice pas. Neuronul A nu mai poate conta pe greșeala lui B, iar Neuronul C trebuie să învețe să lucreze independent. Acest lucru forțează fiecare neuron să devină **mai robust** și să nu se bazeze pe tipare accidentale (zgomot) create de activările altor neuroni.

### Concept Dificil: Injectarea de Zgomot și Finețea (Smoothness)

- **Context:** De ce injectarea de zgomot în straturi duce la *smoothness*?
- *Asta se întâmplă deoarece:* Dacă o rețea trebuie să funcționeze corect, chiar și atunci când jumătate din neuroni sunt dezactivați aleatoriu, ea nu poate dezvolta curbe de decizie extrem de ascuțite sau sensibile. Modelul este forțat să realizeze o cartografiere (*input-output mapping*) care este mai **fină** și mai puțin sensibilă la perturbații mici (sau zgomot).

### Normalizarea $\frac{h}{1-p}$

- **Clarificare:** Normalizarea se face pentru a asigura că, în medie, ieșirea unui neuron are aceeași valoare așteptată, indiferent dacă *dropout* este aplicat sau nu.
- **Exemplu:** Dacă $p = 0.5$ (50% șanse de dezactivare), un neuron care rămâne activ va avea o intrare către stratul următor redusă cu 50% în medie. Pentru a compensa, activările neuronilor "supraviețuitori" sunt împărțite la $1-p = 0.5$, adică **multiplicate cu**

**2**. Această normalizare asigură că suma totală așteptată de activări către stratul următor rămâne aceeași ca înainte de *dropout*.

---

# 4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația cu $\ell_2$ Regularization (Lecția 2.4):** Deși ambele luptă împotriva *overfitting*-ului, $\ell_2$ o face prin penalizarea magnitudinii ponderilor, în timp ce *Dropout* o face prin **perturbarea aleatorie a rețelei** în timpul antrenării, forțând neuronii să lucreze independent.
- **Implicația asupra Bias/Variance Trade-Off:** Deoarece *deep neural networks* au în mod natural o **varianță ridicată** (din cauza flexibilității extreme), *Dropout* este o tehnică esențială pentru a reduce această varianță și a îmbunătăți generalizarea.
- **Tehnici Complementare:** *Dropout* și $\ell_2$ sunt adesea folosite **împreună** în practică pentru a exploata beneficiile ambelor forme de regularizare.

---

# 5. EXERCIȚII ȘI REZOLVARE

## Exercițiul D.5: Aplicarea și Normalizarea Dropout

Presupunem că antrenăm un MLP cu un strat ascuns, iar probabilitatea de *dropout* este setată la $p = 0.25$. Ieșirea activării unui neuron ascuns este $h = 4$.

1. Care este probabilitatea ca acest neuron să fie **reținut** (să nu fie dezactivat)?
2. Dacă neuronul este reținut, care este valoarea activării normalizate $h'$ care va fi transmisă stratului următor?
3. Dacă am avea 100 de neuroni într-un strat ascuns și aplicăm *dropout* cu $p = 0.25$, câți neuroni am aștepta să fie dezactivați?

**Rezolvare D.5:**

1. **Probabilitatea de reținere:** Probabilitatea de dezactivare este $p = 0.25$. Probabilitatea de reținere (să fie activ) este $1 - p = 1 - 0.25 = \mathbf{0.75}$.
2. **Valoarea activării normalizate $h'$:** Dacă neuronul este reținut (cu probabilitatea $1 - p$), valoarea activării este $\frac{h}{1-p}$.

$$h' = \frac{4}{1 - 0.25} = \frac{4}{0.75} = \frac{4}{3/4} = 4 \times \frac{4}{3} \approx \mathbf{5.33}$$

3. **Numărul așteptat de neuroni dezactivați:** Numărul așteptat este $N \times p$.
   Număr de neuroni dezactivați $= 100 \times 0.25 = \mathbf{25}$ de neuroni.

## Exercițiul D.6: Dropout vs. Test Time

Explicați de ce *Dropout* este folosit **numai** în timpul antrenării și este dezactivat în timpul testării (*at test time*). Ce ar fi necesar dacă *Dropout* ar fi aplicat în timpul testării?

**Rezolvare D.6:**

1. **Dezactivarea în Timpul Testării:** *Dropout* este dezactivat în timpul testării deoarece antrenarea cu *dropout* poate fi văzută ca antrenarea unui ansamblu (ensemble) de rețele mai mici. La testare, dorim să folosim capacitatea deplină a modelului antrenat (incluzând toți neuronii și ponderile învățate) pentru a obține o predicție cât mai stabilă și mai precisă.
2. **Necesitatea Normalizării Suplimentare (Dacă ar fi aplicat la Test):** Dacă *dropout* ar fi aplicat în timpul testării, ieșirea rețelei ar varia aleatoriu. În plus, dacă nu am aplica normalizarea ($\frac{h}{1-p}$) în timpul antrenării (caz neconvențional), atunci ar trebui să facem o normalizare la momentul testării prin **scalarea ponderilor** cu $1-p$ pentru a menține valoarea așteptată constantă. Totuși, în practica standard, normalizarea se face pe activări în timpul antrenării, iar *dropout* este pur și simplu dezactivat la testare.

# 2.6

Absolut! Am ajuns la inima antrenării rețelelor neuronale. Secțiunea 2.6 ne dezvăluie mecanismul matematic prin care rețelele profunde învață: **Propagarea Înainte, Propagarea Înapoi și Grafurile Computaționale**. Aceasta este o lecție esențială pentru a înțelege ce se întâmplă *sub capotă* în algoritmii de *Deep Learning*.

---

# Ghid de Studiu Complet: 2.6 Propagarea Înainte, Propagarea Înapoi și Grafurile Computaționale

## 1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Această lecție se concentrează pe detaliile procesului de antrenare a rețelelor neuronale, care se bazează pe alternarea a două faze cheie: **Propagarea Înainte (*Forward Propagation*)** și **Propagarea Înapoi (*Backward Propagation* sau *Backpropagation*)**.

În esență, *Forward Propagation* calculează predicția și pierderea, în timp ce *Backpropagation* calculează toți **gradienții** necesari pentru a actualiza ponderile modelului. Deși *framework*-urile moderne de *Deep Learning* (precum PyTorch sau TensorFlow) automatizează majoritatea acestor calcule (*automatic differentiation*), este crucial să înțelegem mecanismul *backpropagation* care folosește **regula lanțului (*chain rule*)** din calculul diferențial pentru a calcula derivații parțiale în ordine inversă, de la ieșire spre intrare. Vizualizarea acestor calcule se realizează cu ajutorul **grafurilor computaționale (*computational graphs*)**.

**Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?**

1. **Ciclul de Antrenare:** Antrenarea rețelelor neuronale implică alternarea între **propagarea înainte**, **propagarea înapoi** și **actualizarea parametrilor** (folosind gradienții).
2. **Propagarea Înainte:** Reprezintă calculul și stocarea variabilelor intermediare (inclusiv ieșirile) ale rețelei, de la stratul de intrare la stratul de ieșire.
3. **Propagarea Înapoi:** Este metoda de calcul a **gradientului** parametrilor rețelei neuronale (ponderile $\mathbf{W}^{(1)}$ și $\mathbf{W}^{(2)}$).
4. **Mecanismul *Backpropagation*:** Se bazează pe **regula lanțului** (*chain rule*) din calculul diferențial și traversează rețeaua în **ordine inversă** (de la pierdere la intrări).
5. **Memorie:** *Backpropagation* **refolosește valorile intermediare stocate** din faza de propagare înainte (pentru a evita calculele duplicate), ceea ce face ca antrenarea rețelelor neuronale să necesite semnificativ mai multă memorie decât simpla predicție.

---

# 2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

## 2.6 Propagarea Înainte, Propagarea Înapoi și Grafurile Computaționale

- Deși *framework*-urile moderne oferă funcția de *backpropagation* automat, înțelegerea modului în care sunt calculați gradienții este esențială pentru o înțelegere profundă a *deep learning*-ului.
- Expunerea se concentrează pe un **MLP cu un singur strat ascuns** cu **regularizare $\ell_2$ (weight decay)**.

### Propagarea Înainte (*Forward Propagation*)

- Propagarea înainte se referă la calculul și stocarea **variabilelor intermediare** (inclusiv ieșirile) ale unei rețele neuronale, în ordine, de la stratul de intrare la stratul de ieșire.
- **Modelul Simplificat (fără bias):**
  - Intrarea este $\mathbf{x} \in \mathbb{R}^d$.
  - Variabila intermediară (scorul liniar al stratului ascuns): $= \mathbf{W}^{(1)}\mathbf{x}$.
  - Vectorul de activare ascuns (aplicarea funcției  element-wise): $= ()$.
  - Vectorul de ieșire (scorul liniar al stratului de ieșire): $\mathbf{o} = \mathbf{W}^{(2)}$.
- **Funcția de Pierdere (Loss Function):** Presupunând pierderea pătratică (squared error loss), pierderea de predicție $L$ pentru un singur exemplu este:

$$L(\mathbf{o}, \mathbf{y}) = \frac{1}{2}(\mathbf{o} - \mathbf{y})^\top(\mathbf{o} - \mathbf{y})$$

- **Termenul de Regularizare ($\ell_2$):**

$$= \frac{\lambda}{2}\left(||\mathbf{W}^{(1)}||_F^2 + ||\mathbf{W}^{(2)}||_F^2\right)$$

- **Funcția Obiectiv (Objective Function)** $J$**:** Pierderea totală regularizată:

$$J = L +$$

## Grafurile Computaționale (*Computational Graphs*)

- Grafurile computaționale ajută la vizualizarea dependențelor operatorilor și variabilelor în cadrul calculului.
- În Figura 13:
  - **Pătratele** denotă **variabilele** (ex: $\mathbf{x}, , , L, J$).
  - **Cercurile** denotă **operatorii** (ex: $\times, , +$).
  - Săgețile (care ilustrează fluxul de date) merg în principal spre dreapta și în sus.

## Propagarea Înapoi (*Backpropagation*)

- *Backpropagation* este metoda de calcul a **gradientului** parametrilor rețelei neuronale (greutățile $\mathbf{W}^{(1)}$ și $\mathbf{W}^{(2)}$).
- Metoda parcurge rețeaua în **ordine inversă**, de la ieșire (Loss $J$) la stratul de intrare.
- Propagarea înapoi se bazează pe **regula lanțului (*chain rule*)** din calculul diferențial.
  - Dacă $= f(X)$ și $= ()$, derivata în raport cu $X$ este $\frac{\partial}{\partial X} = \text{prod}\left(\frac{\partial}{\partial}, \frac{\partial}{\partial X}\right)$.
- Algoritmul stochează variabile intermediare (derivate parțiale) necesare pe parcursul calculului gradientului.

## Calculul Gradienților (Matematică Detaliată)

**1. Gradientul $\mathbf{W}^{(2)}$ (Stratul de ieșire)**:

- Obiectivul este $\frac{\partial J}{\partial \mathbf{W}^{(2)}} = \frac{\partial L}{\partial \mathbf{W}^{(2)}} + \frac{\partial}{\partial \mathbf{W}^{(2)}}$ (Ecuația 2).
- Termenul de pierdere $L$:

$$\frac{\partial L}{\partial \mathbf{W}^{(2)}} = \frac{\partial L}{\partial \mathbf{o}}^{\top}$$

  (Folosind regula lanțului și simplificând).
  - $\frac{\partial L}{\partial \mathbf{o}}$ (Gradientul pierderii în raport cu scorul de ieșire) este $\mathbf{o} - \mathbf{y}$.
  - Deci, $\frac{\partial L}{\partial \mathbf{W}^{(2)}} = (\mathbf{o} - \mathbf{y})^{\top}$.
- Termenul de regularizare :

$$\frac{\partial}{\partial \mathbf{W}^{(2)}} = \lambda \mathbf{W}^{(2)}$$

  .
- **Gradientul Final al $\mathbf{W}^{(2)}$:**

$$\frac{\partial J}{\partial \mathbf{W}^{(2)}} = (\mathbf{o} - \mathbf{y})^{\top} + \lambda \mathbf{W}^{(2)}$$

.

**2. Gradientul $\mathbf{W}^{(1)}$ (Stratul Ascuns)**:

- Obiectivul este $\frac{\partial J}{\partial \mathbf{W}^{(1)}} = \frac{\partial L}{\partial \mathbf{W}^{(1)}} + \frac{\partial}{\partial \mathbf{W}^{(1)}}$ (Ecuația 4).
- Termenul de pierdere $L$:

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \frac{\partial L}{\partial} \mathbf{x}^{\top}$$

.

  - Gradientul intermediar $\frac{\partial L}{\partial}$ necesită derivarea prin funcția de activare :

$$\frac{\partial L}{\partial} = \frac{\partial L}{\partial} \ '()$$

    (Folosind operatorul de înmulțire element-wise ).
  - $\frac{\partial L}{\partial}$ (Gradientul pierderii în raport cu activarea ascunsă) este $\mathbf{W}^{(2)\top}(\mathbf{o} - \mathbf{y})$.
- Termenul de regularizare :

$$\frac{\partial}{\partial \mathbf{W}^{(1)}} = \lambda \mathbf{W}^{(1)}$$

.

- **Gradientul Final al $\mathbf{W}^{(1)}$:**

$$\frac{\partial J}{\partial \mathbf{W}^{(1)}} = \left(\mathbf{W}^{(2)\top}(\mathbf{o} - \mathbf{y}) \ '()\right)\mathbf{x}^{\top} + \lambda \mathbf{W}^{(1)}$$

## Dependența Reciprocă și Memoria

- Antrenarea rețelelor neuronale implică alternarea *forward propagation* cu *backpropagation* și actualizarea modelului.
- Calculul *forward propagation* (termenul de regularizare ) depinde de valorile curente $\mathbf{W}^{(1)}$ și $\mathbf{W}^{(2)}$.
- Calculul gradientului (*backpropagation*) depinde de valoarea variabilei ascunse , care este dată de *forward propagation*.
- **Consecință Crucială:** *Backpropagation* **refolosește valorile intermediare stocate** (precum  și ) din *forward propagation* pentru a evita calculele duplicate.
- Această necesitate de a reține valorile intermediare face ca antrenarea să necesite **semnificativ mai multă memorie** decât simpla predicție.

---

# 3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

## Termen Tehnic: Regula Lanțului (Chain Rule)

- **Analogie Simplificată (Cascada de Eroare):** Regula lanțului este ca o cascadă în care eroarea curge înapoi. Pentru a calcula cât de mult a contribuit greutatea $\mathbf{W}^{(1)}$ la pierderea finală $J$, trebuie să urmărim calea inversă:
  1. Cât de mult s-a schimbat $J$ din cauza schimbării în ieșirea finală $\mathbf{o}$? ($\frac{\partial J}{\partial \mathbf{o}}$)
  2. Cât de mult s-a schimbat $\mathbf{o}$ din cauza schimbării în activarea ascunsă ? ($\frac{\partial \mathbf{o}}{\partial}$)
  3. Cât de mult s-a schimbat  din cauza schimbării în scorul liniar  (prin funcția )? ($\frac{\partial}{\partial}$)
  4. Cât de mult s-a schimbat  din cauza schimbării în $\mathbf{W}^{(1)}$? ($\frac{\partial}{\partial \mathbf{W}^{(1)}}$)
- Propagarea înapoi este pur și simplu aplicarea iterativă a acestei reguli.

### Grafurile Computaționale

- **Context:** Grafurile computaționale nu sunt un instrument de design, ci o **unealtă de vizualizare** și un instrument conceptual care stă la baza *Automatic Differentiation*.
- *Asta se întâmplă deoarece:* Prin definirea explicită a variabilelor (pătrate) și a operațiilor (cercuri), *framework*-urile moderne știu exact ce trebuie calculat (forward) și, mai important, știu ce trebuie derivat (backward) și în ce ordine inversă.

### Memoria Necesară pentru Antrenare

- **Context:** De ce antrenarea necesită mai multă memorie decât predicția?
- *Asta se întâmplă deoarece:* Pentru a calcula gradientul $\frac{\partial L}{\partial}$, avem nevoie de valoarea efectivă a lui  (variabila intermediară) calculată în timpul propagării înainte. Dacă nu stocăm aceste valori, ar trebui să le recalculăm la fiecare pas de *backpropagation*, ceea ce ar fi ineficient. Mărimea acestor valori intermediare crește cu numărul de straturi și mărimea *batch*-ului, explicând de ce antrenarea rețelelor adânci cu *batch*-uri mari poate duce cu ușurință la erori de memorie (*out of memory errors*).

---

## 4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Conexiunea cu Gradient Descent:** Propagarea înapoi este mijlocul prin care obținem **gradientul** $\nabla_{\mathbf{w}} J$, care este ingredientul esențial pentru actualizarea parametrilor în toți algoritmii de optimizare bazați pe Gradient Descent.
- **Integrarea Regularizării (Weight Decay):** Termenul de regularizare  este inclus în funcția obiectiv $J$. În timpul *backpropagation*, gradientul $\frac{\partial J}{\partial \mathbf{W}}$ este calculat ca suma a doi termeni: gradientul erorii de predicție $\frac{\partial L}{\partial \mathbf{W}}$ și gradientul penalizării $\frac{\partial}{\partial \mathbf{W}}$. Acest lucru confirmă că *weight decay* este aplicat direct la actualizarea gradientului.
- **Rolul $'()$:** Derivata funcției de activare $'()$ este vitală în calculul gradientului stratului ascuns. Proprietățile acestei derivate (ex: dacă este mică, ca la Sigmoid) influențează direct problema *vanishing gradient*.

---

# 5. EXERCIȚII ȘI REZOLVARE

Vom genera exerciții bazate pe structura forward pass (similar exercițiului M.1 din conversația anterioară și Exercițiul 2 din sursa) și pe înțelegerea componentelor gradientului.

## Exercițiul B.3: Identificarea Variabilelor Intermediare și a Ieșirii

Considerăm un MLP cu un singur strat ascuns, fără termen de bias, cu funcția de activare , așa cum este descris în surse.

1. Scrieți succesiunea de operații (*forward pass*) care transformă intrarea $\mathbf{x}$ în scorul de ieșire $\mathbf{o}$, definind variabilele intermediare  și .
2. În contextul *backpropagation*, de ce trebuie să fie stocate valorile intermediare  și ?

**Rezolvare B.3:**

1. **Succesiunea Operațiilor (*Forward Pass*):**
   - Variabila intermediară (scorul liniar al stratului ascuns): $= \mathbf{W}^{(1)}\mathbf{x}$
   - Vectorul de activare ascuns: $= ()$
   - Vectorul de ieșire (scorul final): $\mathbf{o} = \mathbf{W}^{(2)}$
2. **Stocarea Variabilelor Intermediare:** Valorile intermediare  și  trebuie stocate deoarece sunt refolosite în timpul *backpropagation*. Pentru a calcula gradienții straturilor anterioare, cum ar fi $\frac{\partial L}{\partial \mathbf{W}^{(1)}}$, *backpropagation* necesită, printre altele, valoarea $\mathbf{x}$ și valorile  și  care au fost calculate în faza *forward*. Stocarea previne calculele duplicate și este motivul pentru care antrenarea consumă multă memorie.

## Exercițiul B.4: Componentele Gradientului $\mathbf{W}^{(2)}$ (Stratul de Ieșire)

Pentru MLP-ul cu regularizare $\ell_2$ descris în sursă, gradientul final al ponderilor stratului de ieșire $\mathbf{W}^{(2)}$ este dat de (conform Ecuației din surse):

$$\frac{\partial J}{\partial \mathbf{W}^{(2)}} = (\mathbf{o} - \mathbf{y})^\top + \lambda \mathbf{W}^{(2)}$$

1. Identificați componenta din această ecuație care provine direct din penalizarea $\ell_2$ (Weight Decay).
2. Identificați componenta care măsoară eroarea de predicție și care este propagată înapoi.

**Rezolvare B.4:**

1. Componenta care provine din penalizarea $\ell_2$ (Weight Decay) este $\lambda \mathbf{W}^{(2)}$. Această componentă determină *weight decay* propriu-zis.
2. Componenta care măsoară eroarea de predicție și este propagată înapoi este $(\mathbf{o} - \mathbf{y})^\top$. Termenul $(\mathbf{o} - \mathbf{y})$ reprezintă diferența dintre predicția modelului și eticheta țintă, care

este esența erorii de predicție.

# 2.7

Bine ați venit! Suntem acum la o lecție extrem de importantă: **Stabilitatea Numerică și Inițializarea**. Aceasta este lecția care explică de ce, chiar și cu formulele corecte (Propagarea Înainte și Înapoi), rețelele neuronale adânci eșuează adesea, și cum putem preveni aceste eșecuri prin simpla alegere inteligentă a valorilor de start.

---

# Ghid de Studiu Complet: 2.7 Stabilitatea Numerică și Inițializarea

## 1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 2.7 subliniază importanța crucială a **schemei de inițializare** a parametrilor rețelelor neuronale (ponderi și bias-uri) pentru menținerea **stabilității numerice** în timpul antrenării. Deși am presupus anterior că inițializarea se făcea după o distribuție pre-specificată, alegerile incorecte în această etapă pot conduce la probleme severe care împiedică antrenarea eficientă.

**Principalele obstacole numerice** sunt problemele **gradientului evanescent (*vanishing gradient*)** și **gradientului exploziv (*exploding gradient*)**. Aceste probleme apar din cauza naturii compuse a calculului gradientului în rețelele adânci (înmulțirea matricilor pe straturi). O inițializare corectă, cum ar fi **Inițializarea Xavier/Glorot**, ajută la stabilizarea varianței activărilor și gradienților de-a lungul straturilor, prevenind astfel eșecul optimizării.

**Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?**

1. **Inițializarea este Crucială:** Alegerea schemei de inițializare joacă un rol semnificativ în învățarea rețelelor neuronale și este crucială pentru menținerea **stabilității numerice**.
2. **Probleme de Instabilitate:** Alegerile slabe de inițializare pot cauza probleme de **gradient exploziv (*exploding gradients*)** sau **gradient evanescent (*vanishing gradients*)** în timpul antrenării.
3. **Cauza Eșecului:** În rețelele profunde, gradientul este un produs de $L - I$ **matrici**. Dacă valorile proprii (*eigenvalues*) inițiale ale acestor matrici sunt prea mari sau prea mici, produsul lor poate deveni **foarte mare sau foarte mic**.
4. **Simetria Ponderilor (Un-Igienică):** Inițializarea tuturor ponderilor dintr-un strat ascuns la **aceeași valoare constantă** $c$ face ca toți neuronii ascunși să aibă aceeași activare și gradient, ducând la o simetrie care nu va fi spartă de Gradient Descent.
5. **Inițializarea Xavier/Glorot:** Această metodă oferă o euristică (o soluție practică) care setează varianța ponderilor astfel încât să se mențină constantă varianța activărilor și varianța gradienților, folosind numărul de intrări ($n_{\text{in}}$) și ieșiri ($n_{\text{out}}$) ale stratului.

---

# 2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

## 2.7 Stabilitatea Numerică și Inițializarea

- Până acum, toate modelele discutate au necesitat inițializarea parametrilor conform unei distribuții pre-specificate. Am luat această inițializare ca fiind garantată.
- Alegerea funcției de activare și modul în care inițializăm parametrii determină cât de rapid converge algoritmul de optimizare.
- Alegerile greșite pot duce la **gradient exploziv** sau **gradient evanescent** în timpul antrenării.

## Instabilitatea Numerică în Rețelele Profunde

- **Modelul Rețelei Adânci:** Se consideră o rețea adâncă cu $L$ straturi, intrare $\mathbf{x}$ și ieșire $\mathbf{o}$.
  - Variabila ascunsă a stratului $l$ este $^{(l)}$ (unde $^{(0)} = \mathbf{x}$).
  - Rețeaua este exprimată ca o compoziție de funcții: $\mathbf{o} = f_L(f_{L-1}(\dots f_1(\mathbf{x})))$.
- **Calculul Gradientului (Propagarea Înapoi):** Gradientul $\frac{\partial \mathbf{o}}{\partial \mathbf{W}^{(l)}}$ (pentru stratul $l$) este produsul a $L - l$ **matrici**.
- **Probleme Numerice:** Gradienții sunt susceptibili la aceleași probleme de *underflow* numeric care apar la înmulțirea prea multor probabilități.
- **Problema Serioasă (Inițială):** Matricele intermediare din calculul gradientului $\mathbf{M}^{(l)}$ pot avea o varietate mare de valori proprii (*eigenvalues*).
  - Produsul acestor matrici poate deveni **foarte mare (*very large*)** (exploding gradient) sau **foarte mic (*very small*)** (vanishing gradient).

## Tipuri de Instabilitate a Gradientului

- **Gradient Exploziv (*Exploding Gradient*):** Gradienți excesiv de mari care distrug modelul, făcând ca actualizarea parametrilor să fie prea mare. În acest caz, nu există nicio șansă ca optimizatorul Gradient Descent să convergă.
- **Gradient Evanescent (*Vanishing Gradient*):** Gradienți excesiv de mici. Parametrii se mișcă foarte puțin la fiecare actualizare, făcând învățarea practic imposibilă.

## Cauza Vanishing Gradient: Funcția Sigmoid

- Problema *vanishing gradient* este adesea cauzată de alegerea funcției de activare aplicată după transformarea liniară.
- Funcția Sigmoid $\sigma(x) = \frac{1}{1+\exp(-x)}$ a fost populară istoric.
- **Derivata Sigmoidului:** Derivata Sigmoidului dispare (*vanishes*) atunci când inputurile sunt **mari** și când sunt **mici**.
  - Gradientul atinge un maxim de 0.25 în jurul lui $x = 0$.
  - Când se aplică *backpropagating* prin mai multe straturi, dacă inputul Sigmoidului nu este aproape de zero, produsul total al gradienților va deveni mic, iar gradientul va fi probabil **tăiat (*cut off*)** la un anumit strat.

- **Soluția Modernă:** Din această cauză, **ReLU (Rectified Linear Units)** a apărut ca alegerea implicită (*default choice*) astăzi, fiind mult mai stabilă.

## Problema Simetriei Parametrizării (Initializarea Constantă)

- O altă problemă crucială este simetria inerentă în parametrizarea rețelelor.
- **Exemplu de MLP cu 2 unități ascunse:** Dacă inițializăm toți parametrii stratului ascuns $\mathbf{W}^{(1)}$ la aceeași constantă $c$:
  - În timpul *forward propagation*, ambele unități ascunse produc **aceeași activare**.
  - În timpul *backpropagation*, diferențierea produce un gradient $\frac{\partial \mathbf{o}}{\partial \mathbf{W}^{(1)}}$ ale cărui elemente iau **aceeași valoare**.
  - Iterațiile de Gradient Descent mențin această simetrie, iar rețeaua nu își va realiza niciodată puterea expresivă, comportându-se ca și cum ar avea **o singură unitate ascunsă**.
- **Soluție:** Inițializarea trebuie să **spargă simetria**. Mini-Batch SGD nu ar sparge această simetrie, dar regularizarea Dropout ar face-o.

## Inițializarea Corectă (Xavier/Glorot)

- O modalitate de a atenua problemele de instabilitate este prin **inițializarea atentă**.
- **Principiul Echilibrului:** O inițializare corectă ar trebui să mențină **distribuția la scară** a ieșirii (variabilei ascunse $o_i$) de la un strat la altul.
- **Modelul Liniar (Fără Neliniarități):** Pentru un strat fără neliniaritate, ieșirea $o_i$ este o sumă ponderată: $O_i = \sum_{j=1}^{n_{\text{in}}} W_{ij} X_j$.
- **Formula Varianței (Simplificată):** Asumând că ponderile $W_{ij}$ și intrările $X_j$ au medie zero și sunt independente, varianța ieșirii $o_i$ este:

$$\text{Var}[o_i] = n_{\text{in}} \sigma^2 \gamma^2$$

  - Unde $n_{\text{in}}$ este numărul de intrări, $\sigma^2$ este varianța ponderilor $W_{ij}$, iar $\gamma^2$ este varianța intrărilor $X_j$.
- **Condiția de Stabilitate:** Pentru a menține varianța activărilor constantă (adică $\text{Var}[o_i] = \text{Var}[X_j]$), trebuie să avem $n_{\text{in}} \sigma^2 = 1$.
- **Dilema Dublă:** O analiză similară pentru *backpropagation* ar arăta că varianța gradientului explodează dacă nu avem $n_{\text{out}} \sigma^2 = 1$, unde $n_{\text{out}}$ este numărul de ieșiri. Nu putem satisface ambele condiții simultan.
- **Inițializarea Xavier (Compromis):** O soluție este de a face un compromis:

$$\frac{1}{2}(n_{\text{in}} + n_{\text{out}})\sigma^2 = 1$$

  sau echivalent, varianța ponderilor ar trebui să fie:

$$\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

.

- **Inițializarea Uniformă Xavier:** Dacă ponderile sunt extrase dintr-o distribuție uniformă $U(-a, a)$ (care are varianța $\frac{a^2}{3}$), inițializarea uniformă Xavier sugerează:

$$\mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

.

---

## 3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

### Termen Tehnic: Gradient Exploziv vs. Evanescent

- **Analogie (Ecou):** Gradientul care se propagă înapoi este ca un semnal.
  - **Exploziv:** Dacă semnalul este amplificat de fiecare strat (matrici cu valori mari), se transformă într-un urlet distorsionat, suprascriind orice informație utilă.
  - **Evanescent:** Dacă semnalul este diminuat de fiecare strat (matrici cu valori mici), semnalul moare complet înainte de a ajunge la straturile de început, iar acele straturi nu învață nimic.

### Concept Dificil: Simetria Ponderilor

- **Explicație Simplificată (ELI5):** Imaginați-vă că aveți doi muncitori (neuroni) care trebuie să sape un șanț, dar le dați amândurora exact aceleași instrucțiuni de start (aceleași ponderi). Indiferent de cât de mult sape (câte iterații de Gradient Descent), ei vor rămâne identici, săpând mereu la aceeași adâncime și viteză. Rețeaua ar funcționa ca și cum ar avea un singur muncitor, risipind puterea celuilalt neuron.
- *Asta se întâmplă deoarece:* Gradient Descent actualizează parametrii pe baza derivatelor parțiale. Dacă neuronii sunt identici, derivatele lor vor fi identice, și actualizările lor vor fi identice. Soluția este să pornim de la valori **diferite** (să spargem simetria), de obicei prin inițializare aleatorie.

### Logica Inițializării Xavier/Glorot

- *Asta se întâmplă deoarece:* Scopul inițializării Xavier este de a preveni ca semnalul să se atenueze sau să explodeze pe măsură ce trece prin rețea (în faza *forward*) și de a preveni ca gradientul să se atenueze sau să explodeze în faza *backward*. Soluția de compromis (folosind atât $n_{\text{in}}$, cât și $n_{\text{out}}$) asigură un flux bidirecțional stabil al informației și al gradientului, bazându-se pe proprietăți statistice (varianță).

---

## 4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația cu Gradient Descent (Lecția 1.3):** Inițializarea influențează direct **convergența algoritmului de optimizare**. O inițializare slabă face ca GD să nu poată converge niciodată (gradient exploziv) sau să convergă extrem de lent (gradient evanescent).
- **Relația cu Funcțiile de Activare (Lecția 2.2):** Alegerea funcției de activare este strâns legată de instabilitatea numerică. Utilizarea funcțiilor precum Sigmoid a cauzat istoric problema *vanishing gradient*, motiv pentru care s-a trecut la ReLU, care este mai stabilă.
- **Tehnici Complementare:** Inițializarea corectă (Xavier) este o modalitate de a **mitiga** instabilitatea. Stabilitatea poate fi îmbunătățită suplimentar prin **optimizare atentă** și **regularizare adecvată**.

---

## 5. EXERCIȚII ȘI REZOLVARE

Vom genera exerciții bazate pe conceptele de instabilitate și formula Xavier, conform modelelor din surse (deși sursa *02_09_Exercises.pdf* nu conține exerciții specifice pe inițializare, vom folosi formatul general).

### Exercițiul I.1: Gradientul Evanescent și Funcția Sigmoid

1. Explicați cauza principală (matematică) pentru care funcția Sigmoid contribuie la problema **gradientului evanescent** în rețelele adânci.
2. Ce se întâmplă cu viteza de învățare (actualizarea parametrilor) într-un strat la care ajunge un gradient evanescent?

**Rezolvare I.1:**

1. **Cauza Matematică:** Derivata funcției Sigmoid devine foarte mică (tinde la zero) atunci când inputul este fie **foarte mare**, fie **foarte mic**. Deoarece *backpropagation* implică **înmulțirea** derivatelor de la fiecare strat, înmulțirea repetată a multor valori subunitare (mai mici de 0.25) duce rapid la dispariția gradientului (valoarea devine aproape zero).
2. **Viteza de Învățare:** Un gradient excesiv de mic înseamnă că actualizarea parametrilor este extrem de mică (*hardly move on each update*). Acest lucru face ca învățarea să fie practic **imposibilă** în straturile timpurii ale rețelei.

### Exercițiul I.2: Problema Simetriei Ponderilor

Presupunem că antrenăm un MLP cu 10 unități ascunse, dar din greșeală inițializăm toate ponderile și bias-urile din primul strat ascuns la valoarea zero.

1. Explicați de ce acest MLP se va comporta ca o rețea cu **o singură unitate ascunsă** (dacă ignorăm *dropout*).
2. Ce tehnică de regularizare (discutată în Lecția 2.5) ar putea sparge totuși această simetrie?

**Rezolvare I.2:**

1. **Comportamentul:** Dacă toți parametrii stratului ascuns sunt inițializați la zero sau la aceeași constantă, toate cele 10 unități ascunse vor avea **activări identice** în timpul propagării înainte. În timpul *backpropagation*, toți gradienții asociați acestor unități vor fi, de asemenea, identici. Deoarece Gradient Descent aplică aceeași actualizare identică la toți, simetria nu este niciodată spartă, iar rețeaua nu își realizează puterea expresivă, comportându-se ca și cum ar avea o singură unitate.
2. **Tehnica de Spargere a Simetriei: Dropout** ar putea sparge această simetrie, deoarece dezactivează aleatoriu neuroni diferiți la fiecare pas, forțând unitățile să ia căi diferite de învățare.

## Exercițiul I.3: Aplicarea Inițializării Xavier/Glorot

Un strat ascuns dintr-un MLP primește $n_{\text{in}} = 100$ intrări și trimite $n_{\text{out}} = 50$ ieșiri. Ponderile stratului sunt extrase dintr-o distribuție Gaussiană cu media zero și varianța $\sigma^2$.

1. Care este valoarea optimă a varianței $\sigma^2$ conform principiului Inițializării Xavier (Glorot)?
2. Dacă am fi folosit un MLP superficial (un singur strat, $L = 1$) și am fi dorit doar să menținem varianța *forward* constantă, care ar fi fost condiția pe varianță?

**Rezolvare I.3:**

1. **Varianța Optimă Xavier:** Inițializarea Xavier folosește formula de compromis pentru a stabiliza atât propagarea înainte, cât și pe cea înapoi:

$$\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

.

$$\sigma^2 = \frac{2}{100 + 50} = \frac{2}{150} = \frac{1}{75} \approx \mathbf{0.0133}$$

2. **Condiția Doar Forward:** Condiția pentru a menține varianța activărilor constantă în timpul propagării înainte (fără a considera *backpropagation*) este:

$$n_{\text{in}}\sigma^2 = 1 \quad \text{sau} \quad \sigma^2 = \frac{1}{n_{\text{in}}}$$

.

# 2.8

Bună ziua! Lecția 2.8, **Calculul în Deep Learning (*Deep Learning Computation*)**, servește ca o sinteză și o trecere în revistă a modului în care ingineria software a facilitat dezvoltarea arhitecturilor complexe. Este esențială pentru a înțelege evoluția de la conceptele matematice simple (un singur neuron) la structuri masive (*blocks*) gestionate de *framework*-uri moderne.

# Ghid de Studiu Complet: 2.8 Calculul în Deep Learning (Deep Learning Computation)

## 1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Progresul rapid al *Deep Learning*-ului nu s-ar fi realizat fără **instrumente software excelente**, pe lângă seturile de date uriașe și hardware-ul puternic. Această lecție urmărește evoluția modului în care cercetătorii concep și implementează rețelele neuronale: de la gândirea la nivelul **neuronului individual** la **straturi întregi (*whole layers*)**, și, în cele din urmă, la **blocuri (*blocks*)** mari și reutilizabile. Această trecere la **abstractizări mai grosiere (*coarser abstractions*)** permite prototiparea rapidă a modelelor complexe și evită munca repetitivă, chiar și pentru arhitecturi extrem de adânci (precum ResNet-152, care are sute de straturi). Conceptul de **bloc** este un element fundamental de programare, tratat ca o clasă, care încapsulează logica de propagare înainte (*forward pass*) și necesită (deși automatizată) logica de propagare înapoi (*backpropagation*).

**Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?**

1. **Rolul Instrumentelor Software:** Instrumentele software flexibile, open-source (începând cu **Theano în 2007**), au fost esențiale pentru progres, permițând cercetătorilor să prototipeze rapid.
2. **Evoluția Abstractizării:** Proiectarea rețelelor a trecut de la concentrarea pe **neuroni individuali** la **straturi** (folosind aritmetica vectorizată) și, în prezent, la **blocuri** mai mari.
3. **Conceptul de Bloc:** Un **bloc** este o componentă modulară care poate descrie un singur strat, mai multe straturi sau chiar modelul în întregime, și este reprezentat ca o **clasă** în programare.
4. **Arhitecturi Complexe (Modele pe Bază de Blocuri):** Rețelele complexe (ex: **ResNet-152** cu sute de straturi) folosesc **tipare repetitive de grupuri de straturi (blocks)** care sunt combinate recursiv.
5. **Funcționalitatea Blocului:** Orice bloc trebuie să definească o funcție de **propagare înainte (*forward propagation*)** și să gestioneze parametrii. Funcția de *backpropagation* (calculul gradientului) este de obicei asigurată de **diferențierea automată (*auto differentiation*)** în *framework*-urile moderne.

---

## 2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

### 2.8 Calculul în Deep Learning

- **Factori de Succes:** Progresul *deep learning*-ului se datorează seturilor de date gigantice, hardware-ului puternic și instrumentelor software excelente.

- **Istoric:** Biblioteca **Theano**, lansată în **2007**, a fost inovatoare, oferind instrumente open-source flexibile pentru a prototipa rapid modele, evitând munca repetitivă și menținând totuși capacitatea de a face modificări la nivel scăzut.

## De la Neuron la Strat

- **Neuronul Individual:** Un singur neuron ia un set de intrări, generează o ieșire scalară și are un set de parametri asociat care poate fi actualizat.
- **Stratul de Neuroni:** Odată ce s-a trecut la rețele cu ieșiri multiple, s-a folosit **aritmetica vectorizată** pentru a caracteriza un întreg strat de neuroni.
    - Straturile, similar neuronilor individuali, iau intrări, generează ieșiri și sunt descrise de parametri reglabili (*tunable parameters*).
- **MLP și Straturile:** Chiar și când s-au introdus MLP-urile, modelul păstrează structura de bază în care fiecare strat constituit ingerează intrări (furnizate de stratul anterior), generează ieșiri (intrări pentru stratul următor) și posedă un set de parametri actualizați de semnalul care curge înapoi (*backward*).

## De la Strat la Bloc (Abstractizarea Grosisă)

- Cercetătorii în rețele neuronale au trecut de la gândirea în termeni de neuroni individuali la conceperea rețelelor în termeni de **straturi întregi** și acum, adesea, de **blocuri** mai grosiere.
- Această tranziție este similară cu modul în care designerii de semiconductori au trecut de la specificarea tranzistorilor la circuite logice și apoi la scrierea de cod.
- **Arhitecturi Complexe:** Implementarea unei rețele profunde, precum **ResNet-152** (populară în viziunea computerizată și care posedă **sute de straturi**), ar deveni anevoioasă dacă ar trebui implementată strat cu strat.
- **Definiția Blocului:** Conceptul de **bloc** este introdus pentru a implementa aceste rețele complexe. Un bloc poate descrie:
    - Un singur strat.
    - O componentă constând din straturi multiple.
    - Modelul însuși.
- **Avantajul Blocului:** Flexibilitatea blocului permite ca acesta să fie combinat în artefacte mai mari, adesea **recursiv**, permițând scrierea unui cod compact pentru implementarea rețelelor complexe.

## Blocul din Perspectiva Programării

- Dintr-un punct de vedere al programării, un bloc este reprezentat de o **clasă**.
- Orice subclasă a unui bloc trebuie să definească o funcție de **propagare înainte (*forward propagation*)** care transformă intrarea în ieșire și trebuie să stocheze toți parametrii necesari.
- Unele blocuri nu necesită parametri deloc.

- **Backpropagation Automatizată:** Teoretic, un bloc trebuie să posede și o funcție de *backpropagation* pentru calculul gradienților. Din fericire, datorită funcției de **diferențiere automată** prezentă în *framework*-urile moderne de *deep learning*, dezvoltatorii trebuie să se preocupe doar de **parametri** și de funcția de **propagare înainte**.

---

## 3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

### Termen Tehnic: Abstractizări mai Grosiere (*Coarser Abstractions*)

- **Analogie Simplificată (Construcția cu Lego):** La început, *Deep Learning* era ca și cum ai construi o clădire din cărămizi individuale (neuroni). Apoi, s-a trecut la piese mai mari, pre-asamblate (straturi). Abstractizarea grosisă înseamnă acum să lucrezi cu **blocuri** (componente mari, gata făcute, precum o întreagă secțiune de perete sau un acoperiș).
- *Asta se întâmplă deoarece:* Această abordare modulară permite dezvoltatorilor să se concentreze pe arhitectura generală a rețelei (cum se leagă blocurile), nu pe detaliile de implementare ale fiecărui neuron sau strat individual.

### Concept Dificil: Rolul Theano (Context Istoric)

- **Context:** Theano (2007) a fost inovator pentru că a permis **calculul bazat pe grafuri**.
- *Asta se întâmplă deoarece:* Această tehnică a permis ca operațiile (Propagarea Înainte) să fie definite la nivel abstract (simboluri), permițând apoi sistemului să calculeze automat derivații pentru *backpropagation* (diferențierea automată), eliberând programatorii de sarcina greoaie și predispusă la erori a calculului manual al gradientului.

### ResNet-152 (Sute de Straturi)

- **Context:** Sursa menționează ResNet-152, un model popular în viziunea computerizată care a câștigat competițiile ImageNet și COCO computer vision din **2015**.
- *Asta se întâmplă deoarece:* Exemplul ResNet demonstrează necesitatea absolută a abstracțiilor de tip bloc. Fără ele, gestionarea și implementarea celor **sute de straturi** ar fi nepractică și extrem de eronată.

---

## 4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Conexiunea cu Propagarea Înapoi (Lecția 2.6):** Conceptul de bloc depinde direct de principiul *forward propagation* și *backpropagation*. Deși programatorul definește doar

*forward propagation*, diferențierea automată asigură funcția de *backpropagation* pe baza regulii lanțului.

- **Relația Cauză-Efect (Complexitate și Abstracție):**
  - **CAUZĂ:** Dorința de a construi rețele din ce în ce mai adânci (până la sute de straturi).
  - **EFECT:** Necesitatea de a trece de la abstractizarea de bază (stratul) la o abstractizare superioară (blocul), care poate fi combinat recursiv pentru a gestiona complexitatea arhitecturală.

---

# 5. EXERCIȚII ȘI REZOLVARE

Voi genera exerciții de calcul bazate pe modelul MLP din sursa 02*09_Exercises.pdf, aplicând conceptul de parametrizare și _forward pass* la structuri de tip bloc/strat.

## Exercițiul C.1: Calculul Ieșirii (Forward Pass) pentru un Strat MLP

Definiți un strat MLP pentru clasificare (similar unui singur neuron Logit/Softmax), având $d = 2$ intrări și $q = 2$ ieșiri. Presupunem că funcția de activare $\sigma$ aplicată este transformarea identitate $\sigma(x) = x$ (sau scorurile liniare, Logits).

Pentru input-ul $X = [3 \quad 1]$ (un singur exemplu, $n = 1$), folosiți următorii parametri:

$$W = [2 \quad -1\ 1 \quad 0] \in \mathbb{R}^{2\times 2}, \quad b = [0.5 \quad 1.0] \in \mathbb{R}^{1\times 2}$$

Calculați ieșirea $\mathbf{O}$ a acestui strat: $\mathbf{O} = XW + b$.

**Rezolvare C.1:**

**Pasul 1: Calculul Scorul Liniar $XW$**

$$\mathbf{XW} = [3 \quad 1]\,[2 \quad -1\ 1 \quad 0] = [(3 \cdot 2 + 1 \cdot 1) \quad (3 \cdot (-1) + 1 \cdot 0)]$$

$$\mathbf{XW} = [7 \quad -3]$$

**Pasul 2: Adăugarea Bias-ului $b$**

$$\mathbf{O} = \mathbf{XW} + \mathbf{b} = [7 \quad -3] + [0.5 \quad 1.0]$$

$$\mathbf{O} = [7.5 \quad -2.0] \in \mathbb{R}^{1\times 2}$$

Ieșirea finală a stratului (logits) este $\mathbf{O} = [7.5 \quad -2.0]$.

## Exercițiul C.2: Parametrizarea unui Bloc Complex

Considerăm un "Bloc de Calcul" (*Computational Block*) format din două straturi complet conectate (Fully Connected - FC), similar unei structuri de tip *block* din ResNet.

- **Stratul FC1:** Primește $d = 10$ intrări și are $h_1 = 5$ unități ascunse.
- **Stratul FC2:** Primește ieșirea din FC1 ($h_1 = 5$ intrări) și are $q = 4$ ieșiri.

1. Specificați dimensiunea matricei de ponderi $W$ pentru fiecare strat (FC1 și FC2).
2. Calculați costul total al parametrilor (greutăți și bias-uri) pentru întregul Bloc de Calcul.

**Rezolvare C.2:**

1. **Dimensiunea Matricelor de Ponderi:**
   - $W^{(1)}$ (FC1): Dimensiunea este (intrări $\times$ ieșiri) $\mathbf{10} \times \mathbf{5}$.
   - $W^{(2)}$ (FC2): Dimensiunea este (intrări $\times$ ieșiri) $\mathbf{5} \times \mathbf{4}$.
2. **Costul Total al Parametrilor:** Costul total este suma ponderilor (W) și a bias-urilor (b) pentru ambele straturi.
   - **FC1:** Ponderi ($10 \times 5 = 50$) + Bias-uri ($h_1 = 5$) $55$ parametri.
   - **FC2:** Ponderi ($5 \times 4 = 20$) + Bias-uri ($q = 4$) $24$ parametri.
   - **Total parametri în Bloc:** $55 + 24 = \mathbf{7}$ parametri.

## Exercițiul C.3: Conceptualizarea Blocului și Backpropagation

Dacă un cercetător definește "Blocul de Calcul" din Exercițiul C.2 ca o singură clasă într-un *framework* modern de *deep learning*:

1. Ce funcție esențială trebuie să definească cercetătorul în acea clasă (conform sursei)?
2. Care este motivul pentru care cercetătorul nu trebuie să se preocupe de implementarea explicită a funcției de *backpropagation*?

**Rezolvare C.3:**

1. Cercetătorul trebuie să definească în mod explicit funcția de **propagare înainte (*forward propagation function*)**, care descrie cum sunt transformate intrările în ieșiri (adică, modul în care $F1$ $F2$ se calculează).
2. Cercetătorul nu trebuie să implementeze *backpropagation* deoarece *framework*-urile moderne de *deep learning* includ o funcție de **diferențiere automată (*auto differentiation feature*)** care calculează automat toți gradienții necesari (folosind regula lanțului) odată ce logica *forward* și parametrii au fost definiți.

# 2.9

Absolut! Având în vedere toată materia parcursă, de la bazele liniare până la detaliile computaționale ale rețelelor neuronale profunde, am pregătit o serie de exerciții numerice și de calcul, inspirate direct din stilul și complexitatea problemelor găsite în sursele dumneavoastră.

Vom acoperi Regresia Liniară, Gradient Descent, Softmax, MLP (Propagare Înainte și Parametrizare) și Regularizare ($\ell_2$ Weight Decay și Dropout).

# Exerciții Numerice și de Calcul pe Modele Liniare și Rețele Neuronale Profunde

## A. Modele Liniare și Optimizare (Regresie Liniară și Gradient Descent)

### Exercițiul A.3: Aplicarea Batch Gradient Descent (BGD)

Presupunem că avem un set de date format din trei puncte în planul 2D:

$$\text{Dataset } = (x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), (x^{(3)}, y^{(3)}) = (1,1), (2,-1), (3,2)$$

Vrem să estimăm modelul de regresie liniară $\hat{y} = w_1 x + w_0$.

Folosind **Batch Gradient Descent**, asumați că ponderile inițiale sunt $\mathbf{w}^{(0)} = [w_0 \; w_1] = [0.5 \; 0.5]$ și rata de învățare este $\eta = 0.1$.

Calculați noua valoare a ponderilor $\mathbf{w}^{(1)}$ după o singură iterație.

**Formule necesare (Conform conversației anterioare și surselor):**

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \frac{1}{n} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$$

$$\mathbf{w}^{(1)} \leftarrow \mathbf{w}^{(0)} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}^{(0)})$$

**Rezolvare A.3:**

**Pasul 1: Construirea matricelor $\mathbf{X}$, $\mathbf{y}$ și $\mathbf{w}^{(0)}$ (n=3):**

$$\mathbf{X} = [1 \quad 1\,1 \quad 2\,1 \quad 3], \quad \mathbf{y} = [1 \;\; -1\;2], \quad \mathbf{w}^{(0)} = [0.5 \; 0.5]$$

**Pasul 2: Calculul predicțiilor inițiale $\hat{\mathbf{y}}^{(0)}$ și al erorii $\mathbf{e}^{(0)}$:**

$$\hat{\mathbf{y}}^{(0)} = \mathbf{X}\mathbf{w}^{(0)} = [1 \quad 1\,1 \quad 2\,1 \quad 3]\,[0.5 \; 0.5] = [0.5 + 0.5 \; 0.5 + 1.0 \; 0.5 + 1.5] = [1.0 \; 1.5 \; 2.0]$$

$$\mathbf{e}^{(0)} = \hat{\mathbf{y}}^{(0)} - \mathbf{y} = [1.0 \; 1.5 \; 2.0] - [1 \;\; -1\;2] = [0.0 \; 2.5 \; 0.0]$$

**Pasul 3: Calculul gradientului $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}^{(0)})$:**

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}^{(0)}) = \frac{1}{3} \mathbf{X}^\top \mathbf{e}^{(0)} = \frac{1}{3} [1 \quad 1 \quad 1\,1 \quad 2 \quad 3]\,[0.0 \; 2.5 \; 0.0] = \frac{1}{3} [0.0 + 2.5 + 0.0 \; 0.0 + 5.0 + 0.0] = \frac{1}{3}$$

**Pasul 4: Actualizarea ponderilor $\mathbf{w}^{(1)}$:**

$$\mathbf{w}^{(1)} \leftarrow [0.5 \; 0.5] - 0.1\,[0.833 \; 1.667] = [0.5 - 0.0833 \; 0.5 - 0.1667] \approx [0.4167 \; 0.3333]$$

## B. Regresia Softmax (Clasificare Multi-Clasă)

### Exercițiul S.3: Calculul Probabilităților Softmax (Clasificare Multiclass)

Un model Softmax a calculat următoarele scoruri (logits) pentru o instanță în 3 clase mutual exclusive:

- Clasa 1 (Scor): $o_1 = 3$
- Clasa 2 (Scor): $o_2 = 1$
- Clasa 3 (Scor): $o_3 = 0$

Calculați probabilitățile estimate $\hat{p}_1, \hat{p}_2, \hat{p}_3$.

$$\hat{p}_k = \text{softmax}(\mathbf{o}(\mathbf{x}))_k = \frac{\exp(o_k(\mathbf{x}))}{\sum_{j=1}^{q} \exp(o_j(\mathbf{x}))}$$

*Nota: Folosiți o aproximare pentru* $\exp(3) \approx 20.0$, $\exp(1) \approx 2.72$ *și* $\exp(0) = 1$.

**Rezolvare S.3:**

**Pasul 1: Calculul Numitorului (Suma Exponențialelor)**

$$\sum_{j=1}^{3} \exp(o_j) = \exp(3) + \exp(1) + \exp(0) \approx 20.0 + 2.72 + 1.00 = 23.72$$

**Pasul 2: Calculul Probabilităților**

$$\hat{p}_1 = \frac{\exp(3)}{23.72} \approx \frac{20.0}{23.72} \approx 0.843$$

$$\hat{p}_2 = \frac{\exp(1)}{23.72} \approx \frac{2.72}{23.72} \approx 0.115$$

$$\hat{p}_3 = \frac{\exp(0)}{23.72} \approx \frac{1.0}{23.72} \approx 0.042$$

**Pasul 3: Verificarea (Suma totală):**

$$\hat{p}_1 + \hat{p}_2 + \hat{p}_3 \approx 0.843 + 0.115 + 0.042 = 1.000$$

# C. Perceptroni Multistrat (MLP) și Calcul (Forward Pass)

## Exercițiul M.3: Propagare Înainte (Forward Pass) și Număr Parametri

Definiți un MLP pentru regresie, având $d = 3$ intrări, un strat ascuns cu $h = 2$ unități, folosind funcția de activare ReLU, și $q = 1$ ieșire.

Pentru input-ul $X = [1 \quad 0 \quad 1\ 0 \quad 1 \quad 1]$ (două exemple, $n = 2$), folosiți următorii parametri:

$$W^{(1)} = [1 \quad 0\ 0 \quad 1\ 1 \quad -1], \quad b^{(1)} = [1 \quad 0]$$

$$W^{(2)} = [1\ 0], \quad b^{(2)} = [1]$$

1. Calculați ieșirea $\mathbf{O}$ a rețelei (procesul *forward pass*).
2. Specificați numărul total de parametri (greutăți și bias-uri) ai rețelei.

**Formule necesare (Conform surselor):**

$$H = \sigma(XW^{(1)} + b^{(1)})$$

$$O = HW^{(2)} + b^{(2)}$$

$$\sigma(x) = \text{ReLU}(x) = \max(x, 0)$$

**Rezolvare M.3:**

**1. Calculul Ieșirii O:**

**Pasul 1.1: Calculul scorurilor liniare pentru Stratul Ascuns $XW^{(1)}$:**

$XW^{(1)} = [1 \quad 0 \quad 1\ 0 \quad 1 \quad 1]\,[1 \quad 0\ 0 \quad 1\ 1 \quad -1] = [(1+0+1) \quad (0+0-1)\ (0+0+1) \quad (0+1-1)]$

**Pasul 1.2: Adăugarea Bias-ului $b^{(1)}$:**

$$XW^{(1)} + b^{(1)} = [2 \quad -1\ 1 \quad 0] + [1 \quad 0\ 1 \quad 0] = [3 \quad -1\ 2 \quad 0]$$

**Pasul 1.3: Aplicarea Funcției de Activare ReLU:**

$H = \text{ReLU}\left([3 \quad -1\ 2 \quad 0]\right) = [\max(3,0) \quad \max(-1,0)\ \max(2,0) \quad \max(0,0)] = [3 \quad 0\ 2 \quad 0] \in \mathbb{R}^{2\times 2}$

**Pasul 1.4: Calculul ieșirii O:**

$$O = HW^{(2)} + b^{(2)} = [3 \quad 0\ 2 \quad 0]\,[1\ 0] + [1\ 1] = [3\cdot 1 + 0\cdot 0\ 2\cdot 1 + 0\cdot 0] + [1\ 1]$$

$$O = [3\ 2] + [1\ 1] = [4\ 3] \in \mathbb{R}^{2\times 1}$$

**2. Numărul Total de Parametri:**

- Ponderi $W^{(1)}$: $d \times h = 3 \times 2 = 6$.
- Bias-uri $b^{(1)}$: $h = 2$.
- Ponderi $W^{(2)}$: $h \times q = 2 \times 1 = 2$.
- Bias-uri $b^{(2)}$: $q = 1$.
- Total parametri: $6 + 2 + 2 + 1 = \mathbf{11}$.

# D. Regularizare și Stabilitate Numerică (Weight Decay și Dropout)

## Exercițiul R.1: Penalizarea Weight Decay (Regularizarea $\ell_2$)

Considerăm un strat MLP cu matricea de ponderi $W = [2 \quad 1\ 0 \quad -2]$. Presupunem că hiperparametrul de regularizare este $\lambda = 0.5$.

Calculați valoarea numerică a termenului de penalizare $\ell_2$ care ar fi adăugat la funcția de pierdere: $\frac{\lambda}{2}||W||_F^2$.

**Rezolvare R.1:**

**Pasul 1: Calculul Normei Frobenius la Pătrat ($||W||_F^2$)** Norma Frobenius la pătrat este suma pătratelor tuturor elementelor matricei:

$$||W||_F^2 = 2^2 + 1^2 + 0^2 + (-2)^2 = 4 + 1 + 0 + 4 = 9$$

**Pasul 2: Calculul Termenului de Penalizare**

$$\text{Penalizare } = \frac{\lambda}{2}||W||_F^2 = \frac{0.5}{2} \times 9 = 0.25 \times 9 = \mathbf{2.25}$$

## Exercițiul R.2: Inițializarea Xavier/Glorot (Stabilitate Numerică)

Un MLP folosește Inițializarea Xavier (Glorot) pentru a seta varianța ponderilor unui strat ascuns. Acest strat primește $n_{\text{in}} = 200$ intrări și trimite $n_{\text{out}} = 100$ ieșiri. Ponderile sunt extrase dintr-o distribuție Gaussiană cu media zero și varianța $\sigma^2$.

Calculați varianța optimă $\sigma^2$ (Conform formulei Xavier/Glorot).

**Formule necesare (Conform conversației anterioare):**

$$\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

**Rezolvare R.2:**

$$\sigma^2 = \frac{2}{200 + 100} = \frac{2}{300} = \frac{1}{150} \approx \mathbf{0.00667}$$

## Exercițiul R.3: Aplicarea Dropout la Antrenare

Un strat ascuns dintr-un MLP are 400 de neuroni. În timpul antrenării, aplicăm *Dropout* cu o probabilitate de dezactivare $p = 0.4$. Activarea calculată a unui neuron reținut (înainte de normalizare) este $h = 10$.

1. Câți neuroni ne așteptăm să fie dezactivați la un pas dat?
2. Care este valoarea activării normalizate $h'$ care este transmisă stratului următor?

**Rezolvare R.3:**

1. **Număr așteptat de neuroni dezactivați:** Numărul de neuroni dezactivați $= 400 \times p = 400 \times 0.4 = \mathbf{160}$ neuroni.
2. **Valoarea activării normalizate $h'$:** Probabilitatea de reținere este $1 - p = 1 - 0.4 = 0.6$.

$$h' = \frac{h}{1 - p} = \frac{10}{0.6} \approx \mathbf{16.67}$$

Absolut! Am înțeles că doriți mai multe exerciții de tip calcul, bazate pe structura unui **Multi-Layer Perceptron (MLP)**, similar cu **Exercițiul 2** din surse, care definește o arhitectură MLP, calculează ieșirea (*forward pass*) și numărul total de parametri.

Voi genera un nou exercițiu detaliat, concentrându-mă pe aceleași concepte: **Propagarea Înainte (Forward Pass)** și **Calculul Parametrilor** pentru un MLP, folosind funcția de activare **ReLU**.

---

# Exercițiu Tip MLP: Arhitectură și Propagare Înainte (Forward Pass)

## Exercițiul MLP.1: Calculul Ieșirii și Costul Parametrilor

Definiți un MLP pentru regresie, având **2 intrări** ($d = 2$), un singur **strat ascuns cu 3 unități** ($h = 3$) și **1 ieșire** ($q = 1$).

Funcția de activare pentru stratul ascuns este **ReLU** ($\sigma(x) = \max(x, 0)$).

Pentru matricea de intrare $X$ și următorii parametri de ponderi și bias-uri, calculați **ieșirea $\mathbf{O}$ a rețelei** și specificați **numărul total de parametri (greutăți și bias-uri)** pentru rețea.

$$\text{Intrare: } X = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \in \mathbb{R}^{2\times 2}$$

$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 0 & 1 \\ -1 & 2 & 0 \end{bmatrix} \in \mathbb{R}^{2\times 3}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix} \in \mathbb{R}^{1\times 3}$$

$$\mathbf{W}^{(2)} = \begin{bmatrix} 2 \\ 1 \\ -1 \end{bmatrix} \in \mathbb{R}^{3\times 1}, \quad \mathbf{b}^{(2)} = [0] \in \mathbb{R}^{1\times 1}$$

**Formule necesare (Conform surselor):**

- Stratul Ascuns: $H = \sigma(XW^{(1)} + b^{(1)})$
- Stratul de Ieșire: $O = HW^{(2)} + b^{(2)}$

**Rezolvare MLP.1:**

**1. Calculul Total al Parametrilor:**

Calculăm costul parametrilor pentru fiecare strat:

- **Stratul 1 (Intrare $d = 2$, Ascuns $h = 3$):**
  - Ponderi $W^{(1)}$: $d \times h = 2 \times 3 = 6$
  - Bias-uri $b^{(1)}$: $h = 3$
- **Stratul 2 (Ascuns $h = 3$, Ieșire $q = 1$):**
  - Ponderi $W^{(2)}$: $h \times q = 3 \times 1 = 3$
  - Bias-uri $b^{(2)}$: $q = 1$

$$\text{Total parametri} = 6 + 3 + 3 + 1 = \mathbf{13}$$

**2. Calculul ieșirii O (Forward Pass):**

**Pasul 2.1: Calculul scorurilor liniare pentru Stratul Ascuns ($XW^{(1)}$):**

$$\mathbf{XW}^{(1)} = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ -1 & 2 & 0 \end{bmatrix}$$

$$\mathbf{XW}^{(1)} = \begin{bmatrix} (2 \cdot 1 + 1 \cdot (-1)) & (2 \cdot 0 + 1 \cdot 2) & (2 \cdot 1 + 1 \cdot 0) \\ (0 \cdot 1 + 3 \cdot (-1)) & (0 \cdot 0 + 3 \cdot 2) & (0 \cdot 1 + 3 \cdot 0) \end{bmatrix} = \begin{bmatrix} 1 & 2 & 2 \\ -3 & 6 & 0 \end{bmatrix}$$

**Pasul 2.2: Adăugarea Bias-ului $\mathbf{b}^{(1)}$:**

$$\mathbf{XW}^{(1)} + \mathbf{b}^{(1)} = \begin{bmatrix} 1 & 2 & 2 \\ -3 & 6 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\mathbf{XW}^{(1)} + \mathbf{b}^{(1)} = \begin{bmatrix} 1 & 3 & 2 \\ -3 & 7 & 0 \end{bmatrix}$$

**Pasul 2.3: Aplicarea Funcției de Activare ReLU ($\sigma(x) = \max(x, 0)$):**

$$H = \mathrm{ReLU}\left( \begin{bmatrix} 1 & 3 & 2 \\ -3 & 7 & 0 \end{bmatrix} \right)$$

$$H = \begin{bmatrix} \max(1,0) & \max(3,0) & \max(2,0) \\ \max(-3,0) & \max(7,0) & \max(0,0) \end{bmatrix} = \begin{bmatrix} 1 & 3 & 2 \\ 0 & 7 & 0 \end{bmatrix} \in \mathbb{R}^{2\times 3}$$

**Pasul 2.4: Calculul ieșirii finale $\mathbf{O}$:**

$$\mathbf{O} = H\mathbf{W}^{(2)} + \mathbf{b}^{(2)} = \begin{bmatrix} 1 & 3 & 2 \\ 0 & 7 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \\ -1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\mathbf{O} = \begin{bmatrix} (1 \cdot 2 + 3 \cdot 1 + 2 \cdot (-1)) \\ (0 \cdot 2 + 7 \cdot 1 + 0 \cdot (-1)) \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 + 3 - 2 \\ 0 + 7 + 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\mathbf{O} = \begin{bmatrix} 3 \\ 7 \end{bmatrix} \in \mathbb{R}^{2\times 1}$$

Ieșirea rețelei pentru cele două exemple este $\mathbf{O} = \begin{bmatrix} 3 \\ 7 \end{bmatrix}$.